

**ADVANCED RESNET50 – AUTOENCODER  
FRAMEWORK FOR MULTI-CLASS EYE DISEASE  
CLASSIFICATION**

Project Report Submitted in partial fulfilment of the requirements for the  
award of the degree of

**BACHELOR OF TECHNOLOGY**

in

**COMPUTER SCIENCE AND ENGINEERING**

by

**Tadepalli Naga Sri Samanvitha** **22501A05H6**

**Tati Gayathri** **22501A05H7**

**Vemulapalli Gnaneswari** **22501A05J3**

**Siddula Sushruth** **22501A05G8**

Under the guidance of

**Mr. Bala Bhaskara Rao Emani**

**Assistant Professor**



**Department of Computer Science and Engineering**

**Prasad V. Potluri Siddhartha Institute of Technology**

(Permanently affiliated to JNTU-Kakinada, Approved by AICTE)

(An NBA & NAAC A+ accredited and ISO 21001:2018 certified institute)

**Kanuru, Vijayawada – 520 007**

**2025 – 2026**

# **Prasad V. Potluri Siddhartha Institute of Technology**

(Permanently affiliated to JNTU-Kakinada, Approved by AICTE)

(An NBA & NAAC A+ accredited and ISO 21001:2018 certified institute)

**Kanuru, Vijayawada – 520 007**



## **CERTIFICATE**

This is to certify that the project report entitled “**Advanced ResNet50 – Autoencoder Framework for Multi-class Eye Disease Classification**” that is being submitted by Tadepalli Naga Sri Samanvitha (22501A05H6), Tati Gayathri (22501A05H7), Vemulapalli Gnaneswari (22501A05J3), Siddula Sushruth (22501A05G8) in partial fulfilment for the award of the Degree of Bachelor of Technology in Computer Science and Engineering during the academic year 2025-2026.

**Signature of the Guide**  
**Mr. Bala Bhaskara Rao Emani**  
Assistant Professor,  
Prasad V. Potluri Siddhartha  
Institute of Technology,  
Vijayawada

**Signature of the HOD**  
**Dr. A. Jaya Lakshmi**  
Professor and HOD,  
Prasad V. Potluri Siddhartha  
Institute of Technology,  
Vijayawada.

**SIGNATURE OF THE EXTERNAL EXAMINER**

## DECLARATION

We declare that project work entitled “**Advanced ResNet50 – Autoencoder Framework for Multi-class Eye Disease Classification**” is composed by ourselves, that the work contained here in is our own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional except as specified.

Tadepalli Naga Sri Samanvitha (22501A05H6)

Tati Gayathri (22501A05H7)

Vemulapalli Gnaneswari (22501A05J3)

Siddula Sushruth (22501A05G8)

## Acknowledgement

We sincerely thank the following distinguished personalities who have given their advice and support for successful completion of the work.

We are deeply indebted to our most respected guide **Mr. Bala Bhaskara Rao Emani, Assistant Professor**, Department of Computer Science and Engineering for his valuable and inspiring guidance, comments, suggestions and encouragement.

We extend our sincere thanks to **Dr. A. Jaya Lakshmi, Professor & Head** of the department for extending her cooperation and providing the required resources.

We would like to thank our beloved **Principal, Dr. K. Sivaji Babu** for providing the online resources and other facilities to carry out this work.

We're grateful to **Mr. Bala Bhaskara Rao Emani, Assistant Professor** of our department and our esteemed Project Coordinator for his invaluable guidance and efforts. His exceptional leadership, from inception to final review, ensured the project's success.

We would like to extend our sincere gratitude to all the review panel **Dr. P. Sai Kiran, Professor, Dr. S. Phani Praveen, Associate Professor, Dr. J. Rama Devi, Sr. Assistant Professor** for their valuable suggestions in project reviews.

We extend our sincere thanks to all other teaching and non – teaching staff of the department, for their cooperation and encouragement.

Tadepalli Naga Sri Samanvitha (22501A05H6)

Tati Gayathri (22501A05H7)

Vemulapalli Gnaneswari (22501A05J3)

Siddula Sushruth (22501A05G8)

## ABSTRACT

Early detection of eye diseases is crucial for preventing vision loss and improving patient outcomes, especially for conditions like cataract, diabetic retinopathy, and glaucoma. This project presents a deep learning-based approach for automated eye disease classification using a hybrid model that combines a residual autoencoder and ResNet50. The residual autoencoder enhances retinal images by reducing noise while preserving important features, and the processed images are then fed into the ResNet50 network for feature extraction and classification. Fully connected layers generate the final predictions. The model, tested on datasets containing multiple disease classes and normal images, achieves high accuracy in both binary and multi-class classification tasks. In addition, the framework improves robustness against variations in image quality and lighting conditions, making it more reliable for real-world applications. It also reduces the dependency on manual diagnosis, saving time and minimizing human error. Overall, the proposed system demonstrates the effectiveness of deep learning in retinal image analysis and shows strong potential to support ophthalmologists in early diagnosis, large-scale screening, and efficient clinical decision-making.

## LIST OF FIGURES

| <b>FIG.<br/>NO</b> | <b>FIG. NAME</b>                                                                                | <b>Page<br/>No</b> |
|--------------------|-------------------------------------------------------------------------------------------------|--------------------|
| 5.1.1              | Hybrid Residual Autoencoder – ResNet50 Architecture for Retinal Image Analysis                  | 20                 |
| 5.2.1              | Proposed System Architecture                                                                    | 24                 |
| 5.2.2              | Cataract Image                                                                                  | 25                 |
| 5.2.3              | Diabetic Retinopathy Image                                                                      | 25                 |
| 5.2.4              | Glaucoma Image                                                                                  | 25                 |
| 5.2.5              | Normal Image                                                                                    | 26                 |
| 5.2.6              | Proposed System Flow Diagram                                                                    | 29                 |
| 5.2.7              | Use Case Diagram of the Proposed Autoencoder – ResNet50 based Eye Disease Classification System | 30                 |
| 8.2.1              | Binary Classification: Cataract vs Normal – Accuracy Plot                                       | 53                 |
| 8.2.2              | Binary Classification: Cataract vs Normal – Confusion Matrix                                    | 54                 |
| 8.2.3              | Binary Classification: Diabetic Retinopathy vs Normal – Accuracy Plot                           | 54                 |
| 8.2.4              | Binary Classification: Diabetic Retinopathy vs Normal – Confusion Matrix                        | 55                 |
| 8.2.5              | Binary Classification: Glaucoma vs Normal – Accuracy Plot                                       | 55                 |
| 8.2.6              | Binary Classification: Glaucoma vs Normal – Confusion Matrix                                    | 56                 |
| 8.2.7              | Multi- class Classification – Accuracy Plot                                                     | 57                 |
| 8.2.8              | Multi- class Classification – Confusion Matrix                                                  | 57                 |
| A.1                | Model Prediction Output for Cataract Retinal Image                                              | 81                 |
| A.2                | Model Prediction Output for Diabetic Retinopathy Retinal Image                                  | 81                 |
| A.3                | Model Prediction Output for Glaucoma Retinal Image                                              | 82                 |
| A.4                | Model Prediction Output for Normal Retinal Image                                                | 82                 |

## LIST OF TABLES

| Table No | Table Name                         | Page No |
|----------|------------------------------------|---------|
| 7.2.1    | Test Cases                         | 49      |
| 8.2.1    | Binary Classification Results      | 53      |
| 8.2.2    | Multi-class Classification Results | 57      |
| B.1      | Mapping of SDGs                    | 83      |

## **TABLE OF CONTENTS**

| <b>NAME</b>                                          | <b>Page No</b> |
|------------------------------------------------------|----------------|
| <b>CHAPTER 1: INTRODUCTION</b>                       | <b>1</b>       |
| 1.1 Introduction to Project                          | 1              |
| 1.2 Motivation                                       | 3              |
| 1.3 Statement of the Problem and Solution to Problem | 4              |
| 1.4 Objectives                                       | 5              |
| 1.4.1. General Objectives                            |                |
| 1.4.2. Specific Objectives                           |                |
| 1.5 Scope of the Work                                | 6              |
| 1.6 Significance of the Work                         | 6              |
| 1.7 Outlines of the Project                          | 7              |
| <b>CHAPTER 2: BACKGROUND AND LITERATURE REVIEW</b>   | <b>9</b>       |
| <b>CHAPTER 3: SYSTEM ANALYSIS</b>                    | <b>13</b>      |
| 3.1 Existing System                                  | 13             |
| 3.2 Proposed System                                  | 14             |
| 3.3 Feasibility Study                                | 15             |
| 3.3.1 Technical Feasibility                          |                |
| 3.3.2 Operational Feasibility                        |                |
| 3.3.3 Economic Feasibility                           |                |
| <b>CHAPTER 4: SOFTWARE REQUIREMENT SPECIFICATION</b> | <b>17</b>      |
| 4.1 Functional Requirements                          | 17             |
| 4.2 Non – Functional Requirements                    | 18             |



|                                                             |               |
|-------------------------------------------------------------|---------------|
| <b>CHAPTER 5: DESIGN AND METHODOLOGY OF PROPOSED SYSTEM</b> | <b>20</b>     |
| 5.1 System Architecture or Model                            | 20            |
| 5.2 System Design                                           | 24            |
| 5.2.1 Flow Diagram                                          |               |
| 5.2.2 Use Case Diagram                                      |               |
| 5.3 Methodology                                             | 31            |
| 5.3.1 Process Methodology                                   |               |
| 5.3.2 Tools and Techniques                                  |               |
| <br><b>CHAPTER 6: IMPLEMENTATION</b>                        | <br><b>36</b> |
| 6.1 Modules                                                 | 36            |
| 6.2 Description of Sample Code of Each Module               | 40            |
| <br><b>CHAPTER 7: TESTING</b>                               | <br><b>46</b> |
| 7.1 Testing Strategy                                        | 46            |
| 7.2 Test Cases                                              | 49            |
| <br><b>CHAPTER 8: RESULT AND DISCUSSION</b>                 | <br><b>50</b> |
| <br><b>CHAPTER 9: CONCLUSION AND FUTURE WORK</b>            | <br><b>59</b> |
| 9.1 Conclusion                                              | 59            |
| 9.2 Future Work                                             | 60            |
| REFERENCES & BIBLIOGRAPHY                                   | 61            |
| APPENDIX A                                                  | 64            |
| Full Code                                                   | 64            |
| Screenshots                                                 | 81            |
| APPENDIX B                                                  | 83            |
| Mapping of Sustainable Development Goals (SDGs)             | 83            |
| Acceptance Letter                                           | 85            |
| Published Paper                                             | 86            |

# CHAPTER - 1

## INTRODUCTION

### 1.1 Introduction to Project

Vision plays a vital role in human life, enabling individuals to interact with their surroundings, perform daily activities, and maintain independence. Any impairment in vision can significantly affect quality of life and, in severe cases, lead to permanent blindness. Eye diseases such as Cataract, Diabetic Retinopathy, and Glaucoma are among the leading causes of visual impairment worldwide. According to the World Health Organization (WHO), a large percentage of vision loss cases can be prevented through early detection and timely treatment [16]. However, delayed diagnosis, limited access to specialized medical professionals, and the growing number of patients continue to pose serious challenges to healthcare systems, particularly in developing countries.

Traditionally, the diagnosis of eye diseases relies on manual examination of retinal images by ophthalmologists. Although this approach is clinically reliable, it is time-consuming, expensive, and difficult to scale for large population screening programs. Recent studies have explored automated techniques for cataract detection using deep learning and transfer learning models such as MobileNet and ResNet50 to assist ophthalmologists in clinical decision making [1], [2]. Deep learning based systems have also been developed for detecting and grading cataracts using slit-lamp images, demonstrating promising diagnostic performance [3].

Similarly, several research works have focused on automated glaucoma detection using convolutional neural networks (CNNs) and fundus image analysis techniques [4], [5]. In addition, deep learning based approaches have been widely applied for detecting diabetic retinopathy from retinal fundus images, showing significant improvements in diagnostic accuracy compared to traditional image processing methods [6], [7], [8]. These studies demonstrate the potential of artificial intelligence for automated eye disease detection and screening.

Recent advances in Artificial Intelligence (AI) and Deep Learning (DL), particularly Convolutional Neural Networks (CNNs), have significantly improved the ability of computers to analyze complex medical images. CNN-based architectures have been widely used for multi-class classification of ocular diseases using retinal fundus images and transfer learning techniques [9], [10]. Comparative studies of different CNN architectures further confirm that deep learning models can achieve high performance in eye disease classification tasks [14].

Despite these advancements, several challenges remain in applying deep learning models to retinal image analysis. Retinal images often contain noise, uneven illumination, and quality variations caused by different imaging devices and acquisition conditions. These factors can negatively affect classification performance and lead to misclassification, particularly when distinguishing visually similar diseases. Furthermore, advanced deep learning approaches such as ensemble learning models and transformer-based architectures have recently been explored to improve classification accuracy and robustness in eye disease detection systems [11], [13], [15].

To address these challenges, this project proposes an **Advanced ResNet50–Autoencoder framework for multi-class eye disease classification**. The proposed system integrates a lightweight **residual convolutional autoencoder** with a **ResNet50 deep convolutional neural network** to enhance retinal image quality and improve classification accuracy. The autoencoder acts as an image enhancement module that reduces noise while preserving important retinal structures, whereas the ResNet50 model extracts hierarchical features useful for disease detection.

The proposed framework supports both **binary classification and multi-class classification tasks**. Binary classification focuses on distinguishing a specific eye disease from normal retinal images, while multi-class classification enables simultaneous identification of multiple diseases including Cataract, Diabetic Retinopathy, Glaucoma, and Normal classes. By combining residual autoencoder based image enhancement with deep CNN based feature extraction, the system aims to provide an efficient and scalable solution for automated retinal disease detection and computer-aided diagnosis.

## 1.2 Motivation

Eye diseases have become a major global health concern due to their increasing prevalence and potential to cause irreversible vision loss if not detected at an early stage. Conditions such as Cataract, Diabetic Retinopathy, and Glaucoma affect millions of people worldwide and often progress silently without noticeable symptoms during the initial stages. Early diagnosis and timely treatment are therefore crucial to prevent permanent vision damage and to improve the quality of life of affected individuals.

In conventional healthcare systems, eye disease diagnosis primarily depends on manual examination of retinal images by experienced ophthalmologists. Although effective, this approach is highly dependent on the availability of skilled professionals and specialized equipment, which is often limited in rural and underdeveloped regions. Manual diagnosis is also time-consuming and susceptible to human factors such as fatigue and subjective interpretation, making large-scale screening programs difficult to implement and increasing the risk of delayed diagnosis.

To address these challenges, this project is motivated by the need to develop an automated and reliable eye disease classification system using advanced deep learning techniques. The integration of an autoencoder with a deep residual network, such as ResNet50, enhances image quality through noise reduction while enabling accurate feature extraction and stable training. This approach supports early detection, assists ophthalmologists as a decision-support tool, and contributes toward building an efficient, scalable, and accessible solution for eye disease diagnosis, thereby helping to reduce preventable vision loss.

## 1.3 Statement of the Problem and Solution to the Problem

### 1.3.1 Problem Statement

The increasing prevalence of eye diseases such as Cataract, Diabetic Retinopathy, and Glaucoma poses a significant challenge to modern healthcare systems, particularly due to the limitations of traditional diagnostic methods. Manual examination of retinal images by ophthalmologists is time-consuming, costly, and highly dependent on the availability of skilled experts and advanced medical infrastructure. In many regions, especially rural and resource-constrained areas, limited access to specialists leads to delayed diagnosis and increased risk of irreversible vision loss. Furthermore, manual screening is prone to human error and inconsistency, and conventional deep learning models often struggle with noisy retinal images, overfitting, and reduced accuracy when differentiating visually similar eye conditions. These challenges highlight the need for an automated, accurate, and scalable eye disease classification system capable of supporting early detection and assisting clinicians in effective decision-making.

### 1.3.2 Solution

To address the challenges associated with manual diagnosis and limitations of conventional models, this project proposes an automated eye disease classification system based on an integrated **ResNet50–Autoencoder framework**. The proposed solution employs a lightweight residual autoencoder to preprocess retinal images by reducing noise and enhancing important structural features, thereby improving image quality and robustness. The denoised images are then processed by a ResNet50 deep residual network, which effectively extracts discriminative features and performs accurate classification using transfer learning. The system supports both binary and multi-class classification of eye diseases, enabling early detection of Cataract, Diabetic Retinopathy, and Glaucoma with high reliability. By automating the screening process and reducing dependency on manual examination, the proposed approach serves as an efficient, scalable, and cost-effective decision-support tool that assists ophthalmologists and improves accessibility to eye disease diagnosis.

## **1.4 Objectives**

### **1.4.1 General Objectives**

The general objective of this project is to design and develop an automated deep learning-based system for accurate and efficient classification of major eye diseases using retinal images. The project aims to integrate an autoencoder with a ResNet50 convolutional neural network to enhance image quality, improve feature extraction, and support both binary and multi-class eye disease classification. By leveraging advanced deep learning techniques and transfer learning, the system seeks to assist ophthalmologists in early diagnosis, reduce manual screening effort, and provide a scalable and reliable decision-support solution that contributes to improved eye healthcare and prevention of avoidable vision loss.

### **1.4.2 Specific Objectives**

The specific objectives of this project are to preprocess retinal images using a lightweight residual autoencoder for noise reduction and feature enhancement, to implement a ResNet50-based deep learning model for effective feature extraction and classification, and to evaluate the system's performance using standard metrics such as accuracy, precision, recall, F1-score, and confusion matrix analysis. The project further aims to support both binary and multi-class classification of eye diseases, including Cataract, Diabetic Retinopathy, and Glaucoma, while ensuring robustness through data preprocessing, augmentation, and transfer learning. Ultimately, the objective is to develop a reliable, efficient, and scalable eye disease classification framework that can assist clinicians in early detection and decision-making.

## **1.5 Scope of the Work**

The scope of this project is focused on the development of an automated eye disease classification system using deep learning techniques applied to retinal images. The work includes the collection and preprocessing of publicly available retinal image datasets, implementation of a residual autoencoder for image denoising, and utilization of a ResNet50-based convolutional neural network for feature extraction and classification. The system is designed to perform both binary and multi-class classification of eye diseases such as Cataract, Diabetic Retinopathy, and Glaucoma, as well as normal cases. The scope also covers model training, validation, and performance evaluation using standard metrics. However, the project is limited to image-based diagnosis and does not involve clinical validation, real-time hardware deployment, or integration with hospital information systems.

## **1.6 Significance of the Work**

The significance of this project lies in its contribution to improving early detection and diagnosis of major eye diseases such as Cataract, Diabetic Retinopathy, and Glaucoma using advanced deep learning techniques. By integrating a residual autoencoder with a ResNet50-based classification framework, the proposed system enhances retinal image quality and improves classification accuracy. This approach reduces reliance on manual screening, minimizes human error, and supports ophthalmologists by providing a reliable decision-support tool for analyzing retinal images efficiently.

In addition to its clinical relevance, the project demonstrates the practical application of artificial intelligence in medical image analysis and automated healthcare systems. The proposed framework offers a scalable and cost-effective solution that can be utilized in community health centers and mass screening programs, particularly in resource-constrained environments. By promoting accurate and timely diagnosis, the system contributes to preventive healthcare. It helps reduce the burden of avoidable vision loss, highlighting the broader impact of AI-driven solutions in modern healthcare.

## **1.7 Outlines of the Project**

### **1. Introduction**

- Provides an overview of eye diseases and their impact on human vision.
- Discusses the need for automated eye disease classification systems.
- Presents the motivation, problem statement, objectives, scope, and significance of the project.

### **2. Literature Review**

- Reviews existing research and methodologies related to eye disease detection.
- Analyses traditional and deep learning–based approaches used for retinal image classification.
- Identifies limitations of existing systems and research gaps addressed by the proposed work.

### **3. Methodology**

- Describes the dataset used and the preprocessing techniques applied to retinal images.
- Explains the use of autoencoders for image denoising.
- Details the ResNet50-based classification approach for binary and multi-class classification.

### **4. System Architecture**

- Presents the overall architecture of the proposed eye disease classification system.
- Explains the interaction between the preprocessing, the autoencoder, and the ResNet50 modules.
- Describes the data flow and classification process.



## **5. Implementation**

- Discusses the tools, libraries, and frameworks used for system development.
- Explains the implementation of the autoencoder and ResNet50 models.
- Describes the training and validation process.

## **6. Results**

- Presents experimental results and performance evaluation metrics.
- Analyses classification accuracy, precision, recall, and F1-score.
- Compares binary and multi-class classification outcomes.

## **7. Conclusion**

- Summarizes the key findings and contributions of the project.
- Highlights the effectiveness of the proposed approach.
- Discusses possible future enhancements and improvements.

## **8. References**

- Lists all research papers, articles, and resources referred to in the project.
- Ensures proper citation of all referenced materials.

## CHAPTER - 2

### BACKGROUND AND LITERATURE REVIEW

The use of deep learning techniques for automated eye disease classification has been widely explored in recent research due to the growing incidence of vision threatening disorders such as Cataract, Diabetic Retinopathy, and Glaucoma. Researchers have proposed various convolutional neural network–based and transfer learning approaches to improve diagnostic accuracy and reduce reliance on manual examination by ophthalmologists.

Several studies have focused on the application of **MobileNet architectures** for eye disease detection due to their lightweight nature and reduced computational complexity. Sheikh Muhammad Saqib et al. employed MobileNetV1 and MobileNetV2 models to detect cataract and glaucoma from retinal images. Their results showed good accuracy in binary classification tasks; however, performance dropped significantly when multi-class classification was attempted, indicating limitations in distinguishing visually similar eye diseases using lightweight architectures alone [1].

ResNet-based transfer learning approaches have also been investigated for eye disease classification. Salwa Shakir Mahmood et al. utilized ResNet50 for cataract detection and compared its performance with CNN and VGG16 models. Although ResNet50 demonstrated strong feature learning capability, the study highlighted that architectural depth and dataset characteristics significantly influence performance outcomes [2]. These findings suggest that while deep residual networks are powerful, they require careful tuning and preprocessing.

Other researchers explored specialized imaging techniques to enhance diagnostic accuracy. Ki Young Son et al. developed a deep learning model for cataract detection using slit-lamp and retro-illumination images. Their work demonstrated that task-specific imaging and CNN-based classification can achieve high accuracy, but such approaches depend heavily on controlled imaging conditions and may not generalize well to standard fundus images [3].

Glaucoma detection has been addressed using advanced CNN architectures and ensemble methods. Juan J. Gomez-Valverde et al. applied ensemble CNN models and transfer learning techniques using Inception and ResNet architectures, achieving high AUC and sensitivity values. However, the study emphasized that ensemble methods increase computational complexity and resource requirements, limiting their feasibility for large-scale deployment [4].

Hybrid deep learning approaches have also been proposed for glaucoma detection. Researchers combined graph convolutional networks with enhanced U-Net architectures to analyze optic nerve features, achieving improved accuracy compared to traditional CNN models. Although effective, such approaches involve complex architectures and multi-stage processing pipelines, which increase system complexity [5].

Diabetic retinopathy screening has been widely studied using deep transfer learning models. Researchers employed architectures such as ResNet152-V2 and InceptionResNet-V2 to identify diabetic retinopathy from retinal images, reporting high screening accuracy. Despite these successes, such deep models demand significant computational resources and large datasets to maintain consistent performance [6].

VGG-based architectures have also been explored for eye disease classification. Devendra Singh and Dinesh C. Dobhal used VGG16 and VGG19 models in combination with traditional classifiers such as SVM and linear regression. Their results demonstrated that while transfer learning improves accuracy, performance varies depending on the classifier and dataset used [7].

Multi-disease classification systems have been proposed to detect more than one eye condition simultaneously. Some researchers utilized InceptionV3-based frameworks to identify diabetic retinopathy and cataract with high accuracy. However, these systems showed limitations when applied across different datasets, highlighting generalization challenges [8].

Lightweight CNN models were investigated by G. Divya Deepak and Subraya Krishna Bhat to improve classification efficiency in resource-constrained environments. Their work demonstrated that compact CNN-based transfer learning models can achieve competitive accuracy, though often at the cost of reduced robustness compared to deeper networks [9].

Other studies emphasized the importance of transfer learning in improving eye disease classification. Tareq Babaqi et al. showed that CNN performance improved significantly when pre-trained models were fine-tuned on retinal image datasets, reinforcing the value of transfer learning in ophthalmic applications [10].

Ensemble and feature-fusion techniques have also been explored to enhance diagnostic performance. Abdullah et al. employed EfficientNet and DenseNet architectures combined with engineered feature extraction, achieving improved classification results. However, ensemble-based systems increase computational overhead and complexity [11].

Object detection-based approaches have recently been introduced in eye disease analysis. Khalaf and Abdulateef proposed a YOLOv8-based framework for ophthalmic disease classification, demonstrating rapid disease localization and detection. While effective, object detection models require extensive training data and high computational power [12].

Transformer-based and hybrid frameworks have also gained attention. AlMohimeed proposed a multi-stage system using transformer models, feature fusion, and ensemble learning to improve diagnostic accuracy. Although such models outperform single-network approaches, they introduce increased architectural complexity [13].

Comparative studies have been conducted to analyze the strengths and weaknesses of different CNN architectures for eye disease classification. Yaurentius et al. evaluated multiple CNN-based models using fundus images, concluding that no single architecture consistently outperforms others across all disease categories [14].

Recent research has explored attention mechanisms and vision transformers for multi-class diabetic retinopathy classification. Mounika and RaviSankar developed a vision transformer model with attention and squeeze-and-excitation mechanisms, achieving improved feature representation and classification accuracy [15].

Collectively, these studies demonstrate the effectiveness of deep learning and transfer learning in eye disease classification while highlighting persistent challenges such as noise sensitivity, overfitting, high computational cost, and reduced multi-class performance. Most existing works focus on single-disease detection or complex ensemble frameworks, limiting scalability and practical deployment.

Based on the reviewed literature, there is a clear research gap for a unified, efficient, and robust classification framework that enhances image quality while maintaining high classification accuracy. The proposed project addresses this gap by integrating a **lightweight residual autoencoder** for image denoising with a **ResNet50-based deep learning classifier**, supporting both binary and multi-class eye disease classification in a computationally efficient and scalable manner.

## **CHAPTER - 3**

### **SYSTEM ANALYSIS**

#### **3.1 Existing System**

##### **3.1.1 Manual Eye Disease Diagnosis**

In the existing system, eye disease diagnosis is primarily performed through manual examination of retinal images by experienced ophthalmologists. Specialists analyse fundus images to identify symptoms of eye diseases such as Cataract, Diabetic Retinopathy, and Glaucoma based on visual indicators like lens opacity, blood vessel abnormalities, and optic nerve damage. Although this approach is clinically reliable, it is time-consuming, requires specialized expertise, and is difficult to scale for screening large populations, especially in rural and resource-limited regions.

##### **3.1.2 Traditional Computer - Aided Diagnostic Systems**

To support clinicians, traditional computer-aided diagnostic (CAD) systems and basic deep learning models have been introduced. These systems typically rely on handcrafted features or standalone convolutional neural networks to classify retinal images. However, such approaches are highly sensitive to image noise, variations in illumination, and dataset imbalance. Additionally, many existing models focus on single-disease or binary classification and exhibit reduced accuracy when extended to multi-class eye disease classification, limiting their effectiveness in comprehensive screening applications.

##### **3.1.3 Limitations of Existing System**

Existing eye disease diagnosis systems have limitations that affect their efficiency and scalability. Manual diagnosis depends on ophthalmologists, making it time-consuming and prone to inconsistencies due to human factors. In addition, deep learning models are sensitive to image quality variations and limited datasets, which can reduce accuracy. Many systems also focus on binary or single-disease classification, limiting their effectiveness in comprehensive multi-class diagnostic applications.

## 3.2 Proposed System

- **Automated Eye Disease Classification Network**

The proposed system introduces an automated deep learning–based framework designed to classify major eye diseases using retinal images. By eliminating the need for continuous manual examination, the system aims to provide fast, accurate, and consistent diagnostic support for ophthalmologists.

- **Image Preprocessing and Denoising Using an Autoencoder**

A lightweight residual autoencoder is used as a preprocessing module to reduce noise and enhance the quality of retinal images. This step preserves important anatomical structures while minimising irrelevant variations caused by illumination changes or imaging conditions, thereby improving the reliability of feature extraction.

- **Deep Feature Extraction with ResNet50**

The denoised retinal images are fed into a ResNet50 convolutional neural network, which utilizes residual connections to efficiently learn complex and discriminative features. Transfer learning with pre-trained ImageNet weights is employed to improve convergence and generalization, especially when training on limited medical datasets.

- **Support for Binary and Multi-Class Classification**

The proposed system is capable of performing both binary classification (disease vs. normal) and multi-class classification involving Cataract, Diabetic Retinopathy, Glaucoma, and Normal categories. This flexibility enables the system to be used for targeted diagnosis as well as comprehensive screening applications.

- **Performance Evaluation and Reliability**

The system's performance is evaluated using standard metrics such as accuracy, precision, recall, F1-score, and confusion matrix analysis. These metrics provide a detailed assessment of the model's effectiveness and ensure consistent and reliable diagnostic results.

- **Scalable and Decision-Support-Oriented Design**

The proposed framework is designed to be scalable and computationally efficient, making it suitable for deployment in hospitals, clinics, and mass screening programs. By assisting ophthalmologists in early detection and decision-making, the system contributes to improved eye healthcare and the prevention of avoidable vision loss.

### **3.3 Feasibility Study**

#### **3.3.1 Technical Feasibility**

The proposed system is technically feasible as it is developed using well-established and widely supported technologies. The implementation is carried out using the Python programming language along with deep learning libraries such as TensorFlow and Keras, which provide robust tools for model development and training. The use of a pre-trained ResNet50 model through transfer learning reduces training complexity and computational requirements. The system can be executed on standard computing platforms or cloud-based environments such as Google Colab without the need for specialized hardware. Additionally, publicly available retinal image datasets are used, ensuring easy access to training data and reproducibility of results.

#### **3.3.2 Operational Feasibility**

The proposed system is operationally feasible as it is designed to be user-friendly and supportive of clinical workflows. The automated classification process simplifies retinal image analysis and assists ophthalmologists by providing quick and accurate diagnostic insights. The system does not require extensive training for users and can be easily integrated into screening programs in hospitals, clinics, and community health centers. By improving early detection and reducing manual workload, the system enhances operational efficiency and contributes to better patient care.



### **3.3.3 Economic Feasibility**

From an economic perspective, the proposed system is cost-effective and affordable. The project utilizes open-source software tools and libraries, eliminating licensing costs. Since the model can be trained and tested on existing computing resources or free cloud platforms, no additional investment in expensive hardware or infrastructure is required. The automated nature of the system reduces long-term operational costs by minimizing the dependency on manual screening and repeated clinical examinations, making it a financially viable solution for healthcare institutions.

## CHAPTER - 4

### SOFTWARE REQUIREMENT SPECIFICATION

#### 4.1 Functional Requirements

1. **Execution Environment:** The complete execution of the proposed system is carried out using Google Colab, a cloud-based platform that provides free access to computational resources. No local system installation is required for model training or testing.
2. **Operating System:** As Google Colab is a cloud-based service, the underlying operating system is managed by the platform. The system can be accessed from any operating system, such as Windows, Linux, or macOS, through a web browser.
3. **Processor:** Google Colab provides high-performance CPUs that support preprocessing operations and overall system execution. These processors work in coordination with GPU resources for efficient model training.
4. **GPU Requirement:** A GPU is required for training the deep learning models efficiently. Google Colab provides access to GPU accelerators such as NVIDIA GPUs, which significantly reduce training time for the autoencoder and ResNet50 models.
5. **RAM:** Google Colab offers sufficient RAM (typically 12 GB or higher), which is required to load retinal image datasets, train deep learning models, and perform evaluation without memory limitations.
6. **Storage:** The system utilizes Google Drive for cloud storage of datasets, trained models, and output results. This eliminates the need for local hard disk space.
7. **Deep Learning Framework:** The system uses TensorFlow and Keras, which are pre-installed in Google Colab. These frameworks provide the necessary tools to design, train, and evaluate the autoencoder and ResNet50 models.

8. **Autoencoder and ResNet50 Implementation:** Google Colab supports the implementation of convolutional autoencoders and ResNet50 architectures. Transfer learning is applied using pre-trained ImageNet weights, while a custom residual autoencoder is implemented for image denoising.
9. **Image Processing Libraries:** TensorFlow and NumPy are used for image preprocessing tasks such as resizing, normalisation, and dataset handling within the Google Colab environment. The `image_dataset_from_directory` utility provided by TensorFlow is used to load and resize retinal images automatically, ensuring consistency in input dimensions for model training and evaluation.
10. **Dataset Handling:** Retinal image datasets are loaded directly from Google Drive with proper directory structures to support both binary and multi-class classification.
11. **Model Training and Evaluation Tools:** The system uses libraries such as **Scikit-learn** and **Matplotlib** to evaluate performance metrics, including accuracy, precision, recall, F1-score, and confusion matrices, along with visualization of training curves.
12. **Result Validation:** Training progress, accuracy graphs, loss curves, and confusion matrices are generated and displayed within the Google Colab notebook.
13. **Version Control and Collaboration Tools:** Version control systems such as **Git** and platforms like **GitHub** are used to manage code versions and support collaborative development.

## 4.2 Non - Functional Requirements

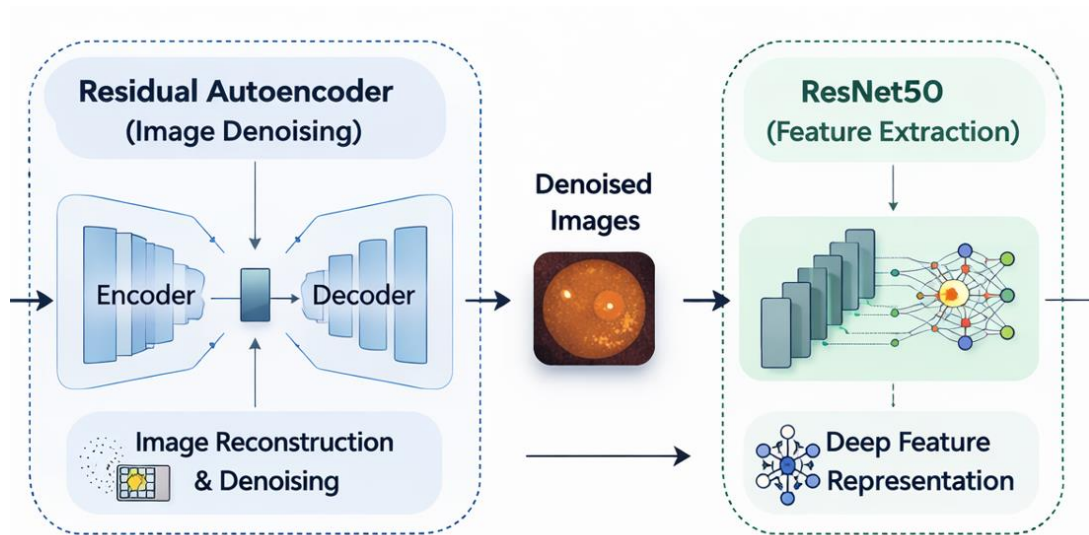
1. **Performance:** The system shall provide efficient performance during training and evaluation by utilizing GPU acceleration available in Google Colab. The classification process must be completed within a reasonable time while handling large retinal image datasets.

2. **Accuracy and Reliability:** The system shall produce reliable and consistent classification results with high accuracy. It must correctly identify eye diseases while minimising false positives and false negatives, ensuring dependable diagnostic support.
3. **Scalability:** The system shall be scalable to accommodate larger datasets and additional eye disease categories in the future without significant changes to the existing architecture.
4. **Availability:** The system shall be available whenever required through the Google Colab platform, provided internet connectivity is available. This ensures ease of access without dependence on local system configuration.
5. **Usability:** The system shall be easy to use for researchers and healthcare professionals with basic knowledge of deep learning. The notebook-based execution provides a clear and interactive interface for model training and evaluation.
6. **Maintainability:** The system shall be easy to maintain and update. Modular implementation of the autoencoder and ResNet50 models allows future enhancements and fine-tuning without affecting the entire system.
7. **Security:** The system shall ensure secure handling of datasets and trained models stored in Google Drive. Access to data is controlled through user authentication provided by Google services.
8. **Portability:** The system shall be portable across different platforms since execution is performed in Google Colab. Users can access and run the system from any operating system through a web browser.

## CHAPTER - 5

### DESIGN AND METHODOLOGY OF PROPOSED SYSTEM

#### 5.1 System Architecture or Model



**Fig. 5.1.1** Hybrid Residual Autoencoder – ResNet50 Architecture for Retinal Image Analysis

The proposed hybrid architecture integrates a Residual Autoencoder for image denoising and reconstruction with a ResNet50 deep convolutional neural network for feature extraction. The model is designed to enhance retinal image quality before extracting deep discriminative features for eye disease classification. The architecture consists of the following major components.

- **Residual Autoencoder (Image Denoising Module)**

The first stage of the hybrid architecture is the Residual Autoencoder, which functions as an image enhancement and denoising mechanism. Autoencoders are unsupervised neural networks that learn to reconstruct input images by compressing them into lower-dimensional representations and then reconstructing them back to their original form.

In this architecture, the autoencoder incorporates residual connections to improve learning efficiency and preserve structural information. Residual connections allow feature information to bypass certain layers, reducing information loss and enabling better reconstruction of important anatomical details. This design improves gradient flow and enhances the stability of deeper networks.

The primary objective of the residual autoencoder is to reduce noise and unwanted variations in retinal images while preserving clinically significant structures such as blood vessels, optic disc regions, and lesion patterns.

- **Encoder Component**

The encoder is the first internal part of the residual autoencoder. It is responsible for extracting essential features from the input image and compressing the image into a compact latent representation.

The encoder consists of multiple convolutional layers that progressively reduce spatial dimensions while increasing feature depth. During this process, low-level features such as edges and textures are captured first, followed by more complex structural patterns. The encoder removes redundant background information and emphasizes disease-relevant patterns.

The output of the encoder is a compressed feature representation, often referred to as the latent space or bottleneck layer. This compact representation contains the most informative features necessary for accurate image reconstruction.

- **Decoder Component**

The decoder is the second internal part of the residual autoencoder. It reconstructs the image from the compressed latent representation generated by the encoder.

The decoder structure mirrors the encoder but performs upsampling operations to restore the image to its original dimensions. Through a series of convolutional and upsampling layers, the decoder reconstructs a cleaner and enhanced version of the original retinal image.

The goal of the decoder is not merely to reproduce the input but to reconstruct it in a way that suppresses noise and artifacts while maintaining important anatomical structures. The reconstruction process ensures that the resulting image is clearer and more suitable for deep feature extraction.

- **Image Reconstruction and Denoising**

The reconstructed output generated by the decoder represents the denoised image. This stage ensures that unwanted noise, illumination inconsistencies, and minor distortions are minimized.

By enhancing image clarity, the denoised output improves the quality of information passed to the next stage of the architecture. This preprocessing enhancement step strengthens the ability of the classification backbone to detect subtle disease-specific features.

The denoised image serves as the refined input to the ResNet50 module.

- **ResNet50 (Feature Extraction Module)**

The second major component of the hybrid architecture is ResNet50, a deep residual convolutional neural network used for feature extraction. ResNet50 is known for its residual learning mechanism, which enables effective training of deep neural networks.

Residual networks introduce shortcut connections that allow the input of a layer to bypass intermediate layers and be added to the output. This structure helps prevent vanishing gradient problems and supports stable learning in deep architectures.

Within this hybrid framework, ResNet50 receives the denoised retinal images and processes them through multiple convolutional layers organized into residual blocks. Each block extracts increasingly complex and abstract features.

- **Residual Blocks in ResNet50**

ResNet50 consists of stacked residual blocks, each containing convolutional layers and identity shortcut connections. These residual blocks enable the network to learn feature transformations while preserving original information through skip connections.

The hierarchical design allows the network to learn:

- ❖ Low-level visual patterns (edges and textures)
- ❖ Mid-level structural features
- ❖ High-level semantic features relevant to eye diseases

This layered feature learning capability makes ResNet50 particularly effective for medical image analysis tasks.

- **Deep Feature Representation**

After processing through multiple residual blocks, ResNet50 generates a deep feature representation of the retinal image. This representation is a high-dimensional feature vector that encodes essential information about structural patterns and abnormalities.

Deep feature representations capture complex spatial relationships and disease-related characteristics. These representations serve as the foundation for classification in the final stage of the network.

The strength of the hybrid architecture lies in combining enhanced image reconstruction from the residual autoencoder with robust hierarchical feature extraction from ResNet50.

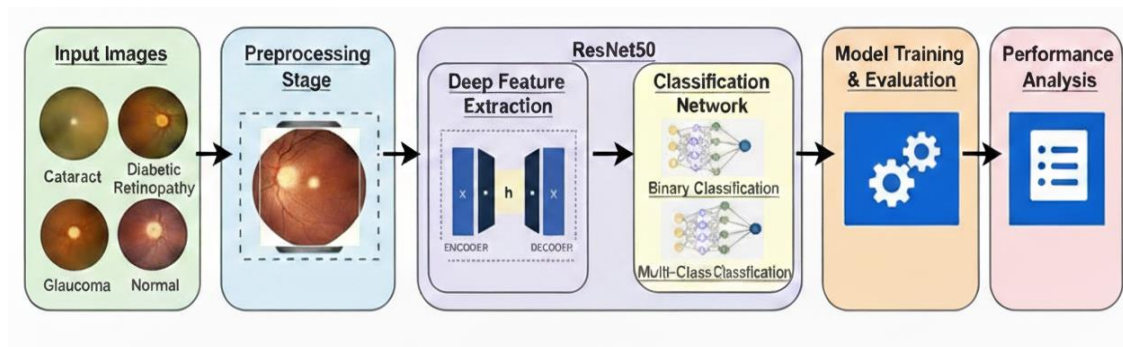


- **Integration of Residual Autoencoder and ResNet50**

The integration of the residual autoencoder with ResNet50 creates a structured two-stage deep learning framework. The autoencoder enhances image quality through reconstruction and denoising, while ResNet50 extracts discriminative features from the refined images.

This integration ensures that the classification backbone operates on cleaner and more informative data. By combining unsupervised reconstruction learning with supervised deep feature extraction, the architecture provides a balanced and robust approach to retinal image analysis.

## 5.2 System Design



**Fig. 5.2.1** Proposed System Architecture

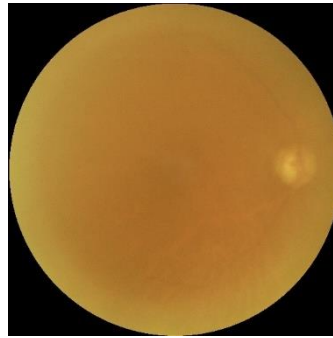
The proposed system follows a structured pipeline beginning with retinal image input and ending with performance analysis. Each phase of the architecture plays a specific role in transforming raw retinal images into meaningful classification outputs.

### 1. Input Images

The system begins with retinal fundus images belonging to four categories: Cataract, Diabetic Retinopathy, Glaucoma, and Normal. These images represent different pathological and healthy conditions of the eye. Retinal images contain complex anatomical structures such as blood vessels, optic discs, and macular regions.

Variations in lighting conditions, imaging devices, and disease severity introduce variability in the dataset. Therefore, raw images cannot be directly fed into deep learning models without preparation.

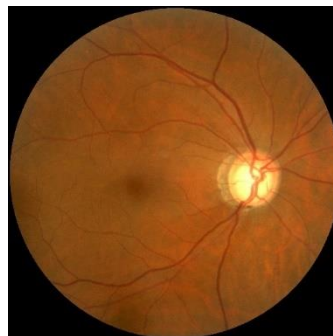
The input stage serves as the data acquisition phase where labeled images are collected and organized for supervised learning.



**Fig.5.2.2** Cataract Image



**Fig.5.2.3** Diabetic Retinopathy Image



**Fig.5.2.4** Glaucoma Image



**Fig.5.2.5** Normal Image

## **2. Preprocessing Stage**

The preprocessing stage prepares retinal images before they are passed into the deep learning model. Since deep neural networks require fixed input dimensions, all images are resized to a uniform resolution. Standardizing image size ensures compatibility with convolutional architectures and improves computational efficiency.

Pixel normalization is also performed to scale intensity values into a consistent range. This step stabilizes gradient updates during model training and prevents numerical instability.

Preprocessing reduces variations caused by differences in image resolution, contrast, and illumination. It ensures that the model focuses on disease-related patterns rather than irrelevant inconsistencies.

## **3. Deep Feature Extraction (Autoencoder + ResNet50)**

### **Autoencoder-Based Feature Enhancement**

Before classification, the images pass through an autoencoder module. An autoencoder is a neural network designed to reconstruct its input. It consists of two main components:

- Encoder
- Decoder

The encoder compresses the input image into a latent representation by extracting essential features while removing redundant information. The decoder reconstructs the image from this compressed representation.

In the proposed architecture, the autoencoder enhances image quality by reducing noise and preserving important anatomical details. This improves clarity and ensures that the classification network receives refined input data.

The reconstructed image produced by the autoencoder serves as an enhanced version of the original retinal image.

#### **4. ResNet50 for Feature Extraction**

After denoising, the refined images are passed into the ResNet50 model. ResNet50 is a deep convolutional neural network that uses residual learning to extract hierarchical features.

Residual connections allow information to bypass certain layers, enabling effective training of deep architectures. This structure improves gradient flow and enhances learning stability.

ResNet50 processes the input image through multiple convolutional layers and residual blocks. During this process, the network learns:

- Low-level features such as edges and textures
- Mid-level structural patterns
- High-level disease-specific characteristics

The output of this stage is a deep feature representation of the retinal image.

#### **5. Classification Network**

The extracted deep features are then forwarded to the classification network. This component consists of fully connected layers that map the feature representation to output categories.

The system supports two types of classification:

- Binary classification (Disease vs Normal)
- Multi-class classification (Cataract, Diabetic Retinopathy, Glaucoma, Normal)

Activation functions convert feature outputs into probability scores for each class. The class with the highest probability is selected as the predicted output.

The classification network transforms learned feature patterns into meaningful diagnostic categories.

## **6. Model Training and Evaluation**

During training, the model learns by adjusting its internal parameters using labeled data. The training process involves forward propagation, loss computation, and backpropagation for weight updates.

Evaluation is performed using validation data to measure the model's ability to generalize to unseen images. Performance metrics provide insight into prediction accuracy and reliability.

This stage ensures that the architecture learns disease-specific patterns effectively and avoids overfitting.

## **7. Performance Analysis**

The final stage involves analyzing the model's predictive performance using validation accuracy and loss metrics obtained during training. These metrics provide insight into the learning behavior of the model and its ability to generalize to unseen data. Performance monitoring helps evaluate the effectiveness of the architecture from input image processing to final classification output.

### 5.2.1 Flow Diagram

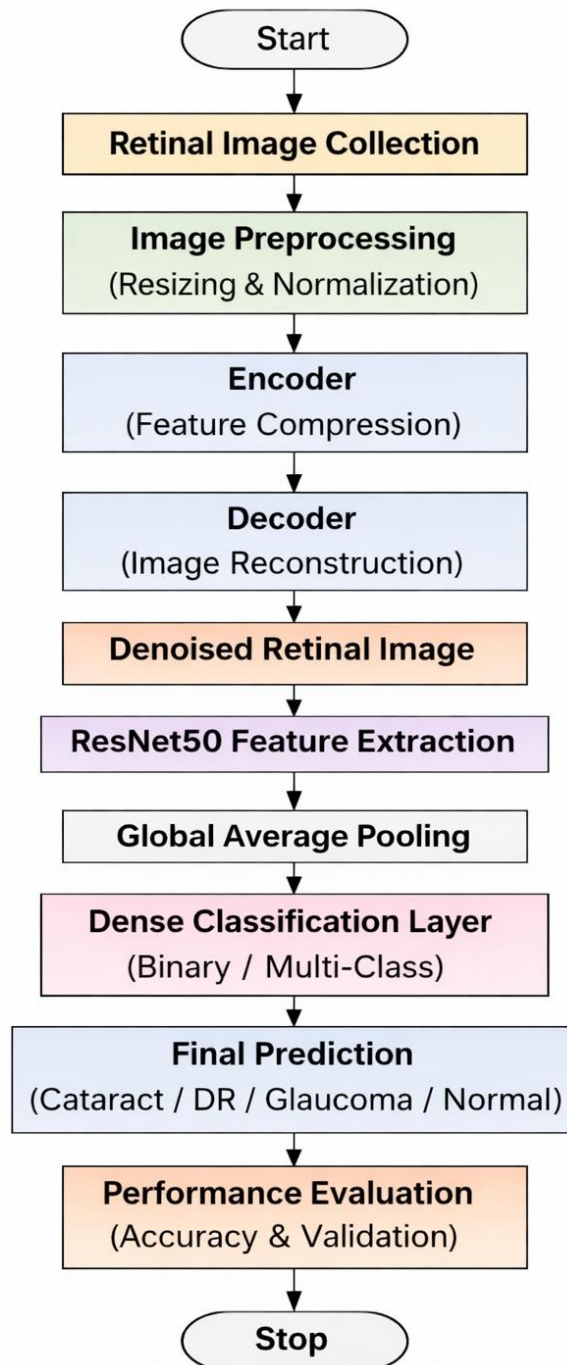
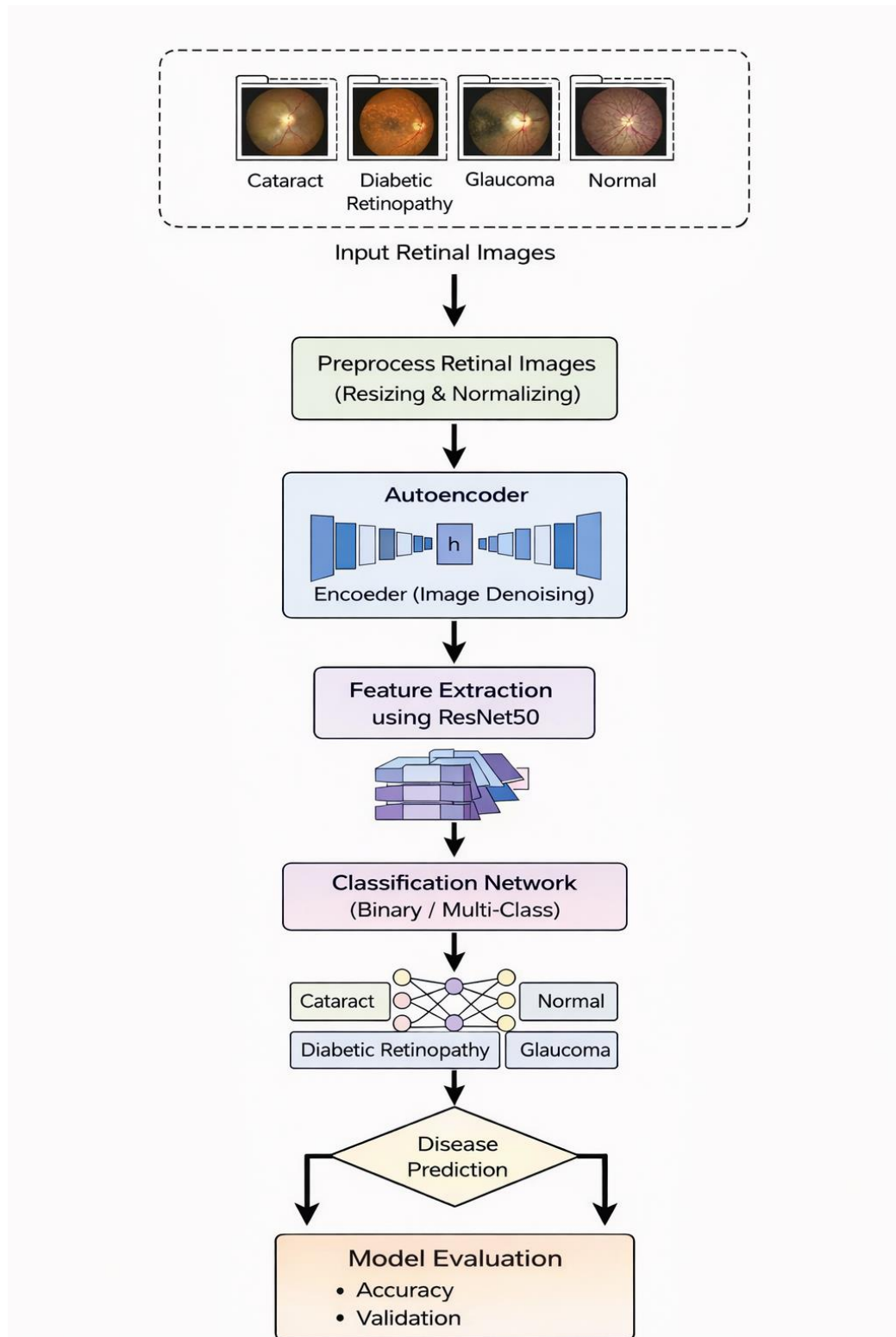


Fig. 5.2.6 Proposed System Flow Diagram

### 5.2.2 Use Case Diagram



**Fig. 5.2.7** Use Case Diagram of the Proposed Autoencoder–ResNet50 Based Eye Disease Classification System

## **5.3 Methodology**

### **5.3.1 Process Methodology**

The proposed eye disease classification system follows a structured methodology integrating image preprocessing, autoencoder-based enhancement, deep feature extraction using ResNet50, and final classification. The overall process model ensures systematic transformation of raw retinal images into accurate disease predictions.

#### **1. Dataset Collection**

The first stage of the methodology involves the collection of retinal fundus images representing various eye conditions. The dataset includes four primary categories: Cataract, Diabetic Retinopathy, Glaucoma, and Normal. Each image is labeled according to its corresponding disease class to support supervised learning.

The dataset is organized into separate class-based directories to maintain proper labelling and structured data handling. This organized arrangement ensures that the learning algorithm correctly associates image features with their respective disease categories.

The quality and diversity of the dataset are critical factors influencing model performance. Therefore, images are selected to include variations in lighting conditions, disease severity, and anatomical differences to improve generalization capability.

#### **2. Data Preprocessing**

Before feeding the images into the deep learning architecture, preprocessing is performed to standardize input data. Since retinal images may vary in size and resolution, all images are resized to a fixed dimension compatible with the network architecture.

Pixel normalization is applied to scale intensity values into a consistent numerical range. This step enhances numerical stability during model training and ensures smooth gradient propagation.



Preprocessing reduces inconsistencies caused by different imaging devices and environmental conditions. It ensures that the model focuses on medically relevant patterns rather than irrelevant background variations.

### **3. Autoencoder-Based Image Denoising**

After preprocessing, the images are passed through a convolutional autoencoder designed to enhance image quality. The autoencoder consists of two main components: an encoder and a decoder.

The encoder compresses the input image into a lower-dimensional latent representation by extracting significant structural features. The decoder reconstructs the image from this compressed representation. During reconstruction, noise and minor distortions are minimized while important anatomical structures are preserved.

The output of this stage is a denoised retinal image. This enhancement improves the clarity of features before deep feature extraction.

### **4. Deep Feature Extraction using ResNet50**

The denoised retinal images are then fed into the ResNet50 deep convolutional neural network. ResNet50 is chosen due to its residual learning mechanism and strong feature extraction capabilities.

The network processes the image through multiple convolutional layers and residual blocks, enabling hierarchical feature learning. Low-level features such as edges and textures are learned in initial layers, while higher layers capture complex disease-related patterns.

Transfer learning is applied to utilize pre-trained weights, improving feature learning efficiency and reducing training time.

## 5. Classification Network

The deep features extracted by ResNet50 are passed to fully connected layers for classification. These layers transform high-dimensional feature vectors into probability scores corresponding to each disease class.

The system supports both binary and multi-class classification tasks. The final output layer produces probability values, and the class with the highest probability is selected as the predicted disease category.

## 6. Model Evaluation

The final stage of the process involves evaluating the model's predictive performance. Training and validation accuracy are monitored to assess learning progress. Loss values are analyzed to ensure proper convergence.

Evaluation ensures that the complete pipeline from dataset collection to final prediction operates effectively and produces reliable results.

### 5.3.2 Tools and Techniques

**1. Hybrid Autoencoder–ResNet50 Architecture:** The proposed system is built using a hybrid deep learning architecture that integrates a convolutional autoencoder with a ResNet50 model. The autoencoder performs image denoising and reconstruction, enhancing retinal image clarity before feature extraction. ResNet50 acts as the deep feature extraction and classification backbone, enabling effective learning of disease-related patterns.

**2. Dataset Collection from Kaggle:** The retinal image dataset used in this project is collected from Kaggle, a widely recognized data science platform. The dataset contains labelled retinal fundus images categorized into Cataract, Diabetic Retinopathy, Glaucoma, and Normal classes. Kaggle provides well-structured datasets suitable for training supervised deep learning models.

**3. Python Programming Language:** The system is developed using Python due to its simplicity and extensive support for machine learning and deep learning libraries. Python enables efficient implementation of neural network architectures and data processing workflows.

**4. Deep Learning Frameworks (TensorFlow and Keras):** TensorFlow and Keras are used to design, train, and evaluate deep learning models. These frameworks provide high-level APIs for constructing convolutional autoencoders and integrating pre-trained models such as ResNet50.

**5. Google Colab Environment:** The entire model training and experimentation process is conducted in Google Colab. This cloud-based platform provides free GPU acceleration, simplifying the execution of computationally intensive deep learning models without requiring high-end local hardware.

**6. Transfer Learning using ResNet50:** Transfer learning is applied by utilising pre-trained ResNet50 weights. This technique improves feature extraction efficiency, reduces training time, and enhances generalisation performance, especially when working with limited medical image datasets.

**7. Image Preprocessing Techniques:** Preprocessing techniques such as image resizing to a fixed resolution and pixel normalization are applied to standardize input data. These steps ensure compatibility with the network architecture and improve numerical stability during training.

**8. GPU Acceleration:** GPU resources available in Google Colab are used to accelerate training and reduce computational time. Deep convolutional networks benefit significantly from parallel GPU processing.

**9. Model Evaluation Techniques:** Model performance is evaluated using training and validation accuracy, along with loss monitoring. These metrics help assess learning progress and ensure proper model convergence.

**10. Version Control and Deployment using GitHub:** The developed code is deployed and managed using GitHub. GitHub provides version control, code management, and collaborative capabilities. It ensures proper documentation, reproducibility, and structured maintenance of the project.

## CHAPTER - 6

### IMPLEMENTATION

#### 6.1 Modules

The implementation of the proposed Residual Autoencoder–ResNet50 based retinal image classification system is organized into several functional modules. Each module performs a specific task in the overall pipeline, enabling systematic processing of retinal images from dataset loading to final disease prediction.

The system is implemented using **Python with TensorFlow and Keras** in the **Google Colab environment**, with **GPU acceleration** used to improve computational efficiency during model training.

#### Dataset Loading Module

This module is responsible for importing the retinal image dataset collected from Kaggle. The dataset is organized into class-specific directories representing different eye disease categories such as **Cataract, Diabetic Retinopathy, Glaucoma, and Normal**.

The module performs the following operations:

- Reading retinal images from structured folders
- Automatic class label assignment based on directory names
- Splitting the dataset into training and validation sets
- Resizing images to **224 × 224 pixels** during loading
- Batch-wise dataset generation for efficient processing

The `image_dataset_from_directory()` function is used to create TensorFlow dataset objects, ensuring proper mapping between images and their corresponding labels.

## Data Preprocessing Module

The preprocessing module prepares the retinal images before they are fed into the neural network architecture.

The main preprocessing operations include:

- Resizing images to a fixed resolution of **224 × 224 pixels**
- Converting images into tensor format compatible with TensorFlow models
- Creating batch datasets for efficient training
- Optimizing the data pipeline using **TensorFlow prefetching**

The prefetch operation enables the system to load the next batch of data while the model processes the current batch, improving overall training performance and reducing input latency.

## Residual Autoencoder Module

The residual autoencoder module is designed to enhance retinal images before feature extraction. This module performs slight reconstruction and correction of the input image to improve feature clarity.

The architecture includes convolutional layers that learn a residual correction applied to the original image through a residual connection. Instead of fully compressing the image into a deep latent representation, the autoencoder learns small corrective features and adds them back to the original input.

This approach preserves important anatomical structures in retinal images while reducing noise and minor distortions. After training, the autoencoder weights are frozen and used as a preprocessing enhancement block within the classification pipeline.

## Feature Extraction Module (ResNet50)

The refined retinal images produced by the autoencoder are passed to the **ResNet50 deep convolutional neural network**, which acts as the backbone feature extractor.

This module performs:

- Loading the **pre-trained ResNet50 model with ImageNet weights**
- Removing the top classification layers
- Freezing backbone weights during initial training
- Extracting deep hierarchical feature representations

ResNet50 processes the retinal images through multiple convolutional and residual blocks, enabling the model to learn complex visual patterns associated with different eye diseases.

### **Classification Module**

The classification module receives deep feature representations generated by the ResNet50 backbone and maps them to disease categories using fully connected layers.

The classification architecture includes:

- **Global Average Pooling** for feature dimension reduction
- **Dense layers with ReLU activation** for feature learning
- **L2 regularization** to prevent overfitting
- **Dropout layers** for improved generalization
- Final output layer for prediction

The system supports **two classification configurations**:

#### **Binary Classification**

Binary classification distinguishes between **healthy retinal images and diseased images**. In this case, the final layer uses **sigmoid activation** to output a probability value representing the predicted class.

#### **Multi-Class Classification**

Multi-class classification identifies specific eye diseases such as **Cataract, Diabetic Retinopathy, Glaucoma, and Normal**. For this configuration, the output layer uses **softmax activation** to generate probability scores across multiple disease categories.

## **Model Training and Validation Module**

This module manages the training process of the complete hybrid architecture.

The training process includes:

- Loss computation using appropriate loss functions
- Backpropagation for gradient computation
- Weight updates using the **Adam optimization algorithm**
- Monitoring training and validation performance

Training and validation accuracy are tracked during each epoch to evaluate the learning behavior of the model and detect potential overfitting.

## **Evaluation and Performance Analysis Module**

After training, the model performance is evaluated using the validation dataset.

The evaluation process measures key metrics such as:

- Classification accuracy
- Training and validation loss
- Learning curve visualization

Accuracy and loss graphs are generated to analyze model convergence and training stability.

## **Prediction Module**

The final module performs prediction on unseen retinal images.

The prediction workflow is:

- Input Retinal Image
- Residual Autoencoder Enhancement
- ResNet50 Feature Extraction
- Dense Classification Layers
- Final Disease Prediction

The system outputs the predicted eye disease category along with the corresponding probability score.



## 6.2 Description of Sample Code of Each Module

This section presents representative code snippets used in the implementation of the proposed Autoencoder–ResNet50 based eye disease classification system. The implementation is carried out using Python with TensorFlow and Keras libraries in the Google Colab environment. The sample code snippets illustrate the key operations performed in each module of the system pipeline.

### Dataset Loading Module

```
DATA_DIR =  
"/content/drive/MyDrive/Dataset/Eye_Disease_Classification_Dataset/Cata  
ract_Normal"  
IMG_SIZE = (224, 224)  
BATCH_SIZE = 32  
EPOCHS = 20  
train_ds = image_dataset_from_directory(  
    DATA_DIR,  
    validation_split=0.2,  
    subset="training",  
    seed=123,  
    image_size=IMG_SIZE,  
    batch_size=BATCH_SIZE,  
    label_mode="binary"  
)  
val_ds = image_dataset_from_directory(  
    DATA_DIR,  
    validation_split=0.2,  
    subset="validation",  
    seed=123,  
    image_size=IMG_SIZE,  
    batch_size=BATCH_SIZE,  
    label_mode="binary"  
)
```

### Code Description

The above code loads retinal images from a directory structure and automatically assigns labels based on class folders. The dataset is split into training and validation sets using a validation split parameter. Images are resized to a fixed dimension of **224 × 224 pixels**, and batches of images are generated to improve computational efficiency during model training.

## Data Preprocessing and Pipeline Optimization

```
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.prefetch(AUTOTUNE)
val_ds = val_ds.prefetch(AUTOTUNE)
```

### Code Description

This step optimizes the data input pipeline by enabling TensorFlow to prepare the next batch of images while the current batch is being processed by the GPU.

## Residual Autoencoder Module

```
def build_autoencoder(img_shape):
    inp = layers.Input(shape=img_shape)
    # small noise correction only
    x = layers.Conv2D(16, 3, activation='relu', padding='same')(inp)
    x = layers.Conv2D(16, 3, activation='relu', padding='same')(x)
    correction = layers.Conv2D(3, 3, padding='same')(x)
    # residual connection
    out = layers.Add()([inp, 0.1 * correction])
    autoencoder = models.Model(inp, out)
    autoencoder.compile(optimizer='adam', loss='mse')
    return autoencoder
autoencoder = build_autoencoder((224,224,3))
```

### Code Description

This module constructs a lightweight residual autoencoder that performs slight denoising of retinal images before feature extraction.

The architecture includes:

- Encoder-like convolution layers
- Residual correction layer
- Residual skip connection
- Image reconstruction output

The residual connection ensures that only small corrections are applied to the original retinal image.

## Autoencoder Training Module

```
autoencoder.fit(  
    train_ds.map(lambda x,y: (x,x)),  
    epochs=5,  
    validation_data=val_ds.map(lambda x,y: (x,x))  
)  
autoencoder.trainable = False
```

### Code Description

The autoencoder is trained in an **unsupervised manner** where the input image is also used as the target output.

After training, the model weights are **frozen** so that the autoencoder acts as a **fixed denoising layer** during classification.

## Feature Extraction Module (ResNet50)

```
base_model = ResNet50(  
    weights="imagenet",  
    include_top=False,  
    input_shape=(224,224,3)  
)  
base_model.trainable = False
```

### Code Description

The pre-trained **ResNet50 deep convolutional network** is used as the backbone feature extractor.

Key configuration:

- ImageNet pretrained weights
- Top classification layer removed
- Transfer learning enabled

This allows the model to leverage **deep hierarchical visual features learned from large-scale datasets**.

## Classification Module

```
inputs = layers.Input(shape=(224,224,3))
# denoised image goes to ResNet
x = autoencoder(inputs)
x = base_model(x)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(128, activation="relu",
                  kernel_regularizer=regularizers.l2(0.01))(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = models.Model(inputs, outputs)
model.compile(
    optimizer=Adam(1e-4),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

## Code Description

This module constructs the **final hybrid architecture**:

- Retinal Image
- Residual Autoencoder
- ResNet50 Feature Extractor
- Global Average Pooling
- Dense Layer
- Dropout Regularization
- Sigmoid Output

The sigmoid activation produces **binary classification predictions**.

## Model Training Module

```
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS
)
```

### Code Description

The hybrid architecture is trained using **binary cross-entropy loss** with the **Adam optimizer**.

Training is performed using:

- Training dataset
- Validation dataset
- Multiple epochs

The training history is stored to analyze model learning behavior.

### Model Evaluation Module

```
train_loss, train_acc = model.evaluate(train_ds)
val_loss, val_acc = model.evaluate(val_ds)
print(f"Training Accuracy (Binary Classification - Cataract vs Normal):
{train_acc*100:.2f}%")
print(f"Validation Accuracy (Binary Classification - Cataract vs
Normal): {val_acc*100:.2f}%")
```

### Code Description

This module evaluates model performance on training and validation datasets by computing:

- Accuracy
- Loss values

These metrics help determine the **generalization capability of the model**.

### Performance Visualization Module

```
plt.figure(figsize=(12,5))
# Accuracy
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Validation')
plt.title('Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
```

```
# Loss
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train')
plt.plot(history.history['val_loss'], label='Validation')
plt.title('Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

### Code Description

This module plots:

- Training Accuracy
- Validation Accuracy
- Training Loss
- Validation Loss

These curves help visualize the learning progress and detect overfitting.

## CHAPTER - 7

### TESTING

#### 7.1 Testing Strategy

The testing strategy defines the systematic approach used to evaluate the correctness, reliability, and performance of the proposed deep learning based eye disease classification system. The primary objective of testing is to ensure that each module of the system functions correctly and that the complete architecture produces accurate predictions when applied to retinal images.

The testing process was performed at multiple levels, including module testing, integration testing, and performance evaluation. These testing procedures help verify both the functionality of individual components and the overall effectiveness of the proposed Autoencoder–ResNet50 architecture.

#### Module Testing

Module testing focuses on verifying the functionality of individual components of the system before they are integrated into the complete architecture. Each module was tested independently to ensure that it performs its intended operation.

For example, the dataset loading module was tested to confirm that retinal images were correctly read from the dataset directories and that labels were properly assigned to each image class. The preprocessing module was verified to ensure that all images were resized to the required input size of  $224 \times 224$  pixels and converted into tensor format suitable for deep learning models.

Similarly, the residual autoencoder module was tested to ensure that it successfully reconstructs input images while preserving important retinal structures. The ResNet50 feature extraction module was validated to confirm that deep features were correctly extracted from the processed images.

## **Integration Testing**

Integration testing was performed after verifying individual modules to ensure that the modules interact correctly when combined into the full system pipeline. The integration process involved connecting the residual autoencoder, ResNet50 backbone, and classification layers into a single hybrid architecture.

During integration testing, the data flow from the input image through each stage of the architecture was examined. The system was tested to ensure that the output of one module correctly serves as the input for the next module without causing compatibility issues or dimension mismatches.

Successful integration testing confirmed that the hybrid architecture operates smoothly from image input to final disease classification.

## **Model Training Verification**

The training process was monitored to ensure that the model learns meaningful patterns from the dataset. Training and validation accuracy were tracked across epochs to verify that the model improves its predictive performance over time.

Loss values were also analyzed to confirm that the training process converges properly. If the training loss decreases while validation loss remains stable, it indicates that the model is learning useful representations without severe overfitting.

This testing step ensures the stability and reliability of the training procedure.

## **Performance Evaluation**

The final stage of testing involves evaluating the predictive performance of the trained model. The model is tested on validation data that was not used during training to measure its ability to generalize to new retinal images.

Performance metrics such as classification accuracy and loss values are used to assess the effectiveness of the model. These metrics provide insights into how well the proposed system can correctly identify different eye disease conditions.



Visualization of accuracy and loss curves further helps analyze the learning behavior of the model and detect potential issues such as underfitting or overfitting.

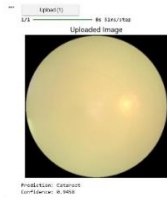
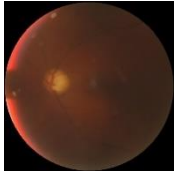
## **Prediction Testing**

Prediction testing is performed by applying the trained model to retinal images and observing the predicted output class. The system processes the input image through the autoencoder enhancement stage, extracts deep features using ResNet50, and finally generates a predicted disease class.

This testing verifies the complete operational workflow of the system and demonstrates that the trained model is capable of performing automated retinal image classification.

## 7.2 Test Cases

**Table. 7.2.1** Test Cases

| S.No | Test Case Name                               | Input                                                                               | Expected Output      | Actual Output                                                                         | Test Status (Pass / Fail) |
|------|----------------------------------------------|-------------------------------------------------------------------------------------|----------------------|---------------------------------------------------------------------------------------|---------------------------|
| 1.   | TC1: Test for Cataract Detection             |    | Cataract             |    | Pass                      |
| 2.   | TC2: Test for Diabetic Retinopathy Detection |   | Diabetic Retinopathy |   | Pass                      |
| 3.   | TC3: Test for Glaucoma Detection             |  | Glaucoma             |  | Pass                      |
| 4.   | TC4: Test for Normal Retinal Classification  |  | Normal               |  | Pass                      |

## CHAPTER - 8

### RESULT AND DISCUSSION

#### 8.1 Performance Evaluation Metrics

Performance evaluation metrics are used to measure the effectiveness of the proposed eye disease classification system. These metrics help determine how accurately the deep learning model can identify different retinal diseases from input images.

##### 1. Accuracy

Accuracy represents the overall proportion of correctly classified images out of the total number of images tested. It is one of the most commonly used metrics for evaluating classification models.

Accuracy is calculated as:

$$\text{Accuracy} = (\text{Number of Correct Predictions}) / (\text{Total Number of Predictions})$$

A higher accuracy value indicates that the model performs well in correctly identifying the retinal disease classes.

##### 2. Precision

Precision measures how many of the images predicted as a particular disease actually belong to that disease class. It indicates the reliability of the model's positive predictions.

Precision is defined as:

$$\text{Precision} = \text{True Positives} / (\text{True Positives} + \text{False Positives})$$

High precision indicates that the model produces fewer false positive predictions.

### **3. Recall**

Recall measures the ability of the model to correctly identify all actual disease cases present in the dataset.

Recall is calculated as:

$$\text{Recall} = \text{True Positives} / (\text{True Positives} + \text{False Negatives})$$

A higher recall value means that the model successfully detects most of the disease cases.

### **4. F1-Score**

The F1-Score is the harmonic mean of precision and recall. It provides a balanced evaluation of the model's performance when both false positives and false negatives are important.

F1-Score is calculated as:

$$\text{F1-Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

A higher F1-score indicates better overall classification performance.

### **5. Confusion Matrix**

A confusion matrix is a tabular representation used to evaluate the classification performance of a model. It shows the number of correct and incorrect predictions made by the model for each class.

The matrix consists of four main components:

- True Positives (TP) – correctly predicted positive cases
- True Negatives (TN) – correctly predicted negative cases
- False Positives (FP) – incorrectly predicted positive cases
- False Negatives (FN) – incorrectly predicted negative cases

## **8.2 Experimental Results**

This section presents the experimental results obtained from the implementation of the proposed Residual Autoencoder–ResNet50 based eye disease classification system. The performance of the model is evaluated using multiple classification tasks including binary classification and multi-class classification of retinal images.

The experiments were conducted using retinal image datasets containing four classes: Cataract, Diabetic Retinopathy, Glaucoma, and Normal. The dataset was divided into training and validation subsets, allowing the model to learn from the training data and evaluate performance on unseen validation images.

The system integrates a residual autoencoder for image refinement and a ResNet50 deep convolutional neural network for feature extraction and classification.

The autoencoder improves image quality through slight residual correction, while the ResNet50 backbone extracts hierarchical features useful for identifying retinal disease patterns.

### **Binary Classification Results**

Binary classification experiments were performed to distinguish each disease class from normal retinal images. These experiments evaluate the ability of the proposed architecture to identify individual diseases.

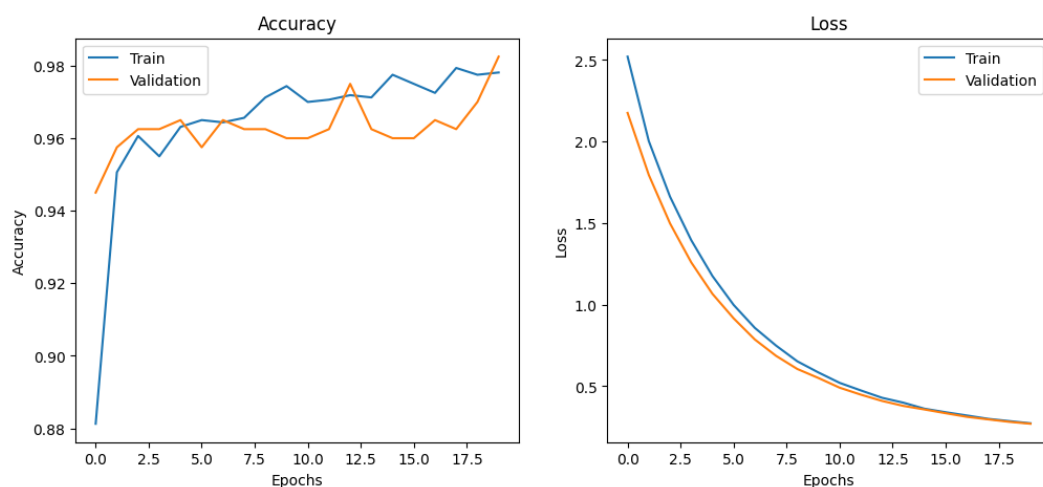
**Table. 8.2.1** Binary Classification Results

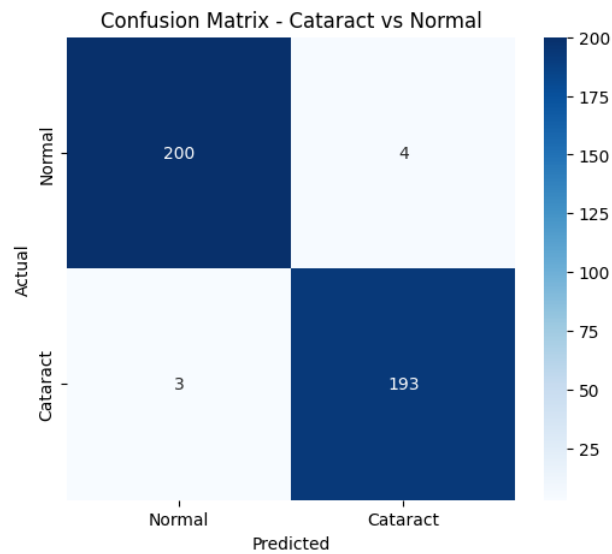
| Disease                        | Class                | Precision | Recall | F1-Score | Train Acc (%) | Test Acc (%) |
|--------------------------------|----------------------|-----------|--------|----------|---------------|--------------|
| Cataract vs Normal             | Normal               | 0.985     | 0.980  | 0.982    | 97.94         | 98.25        |
|                                | Cataract             | 0.979     | 0.984  | 0.982    |               |              |
| Diabetic Retinopathy vs Normal | Normal               | 0.975     | 0.975  | 0.975    | 99.31         | 97.50        |
|                                | Diabetic Retinopathy | 0.974     | 0.974  | 0.974    |               |              |
| Glaucoma vs Normal             | Normal               | 0.959     | 0.813  | 0.880    | 90.00         | 88.75        |
|                                | Glaucoma             | 0.832     | 0.964  | 0.893    |               |              |

### Cataract vs Normal

The model achieved high classification performance in distinguishing cataract images from normal retinal images. After training the model for multiple epochs, the system achieved approximately 97.9% training accuracy and 98.2% validation accuracy.

The high validation accuracy indicates that the model effectively learned discriminative features associated with cataract conditions without significant overfitting.

**Fig. 8.2.1** Binary Classification: Cataract vs Normal – Accuracy Plot

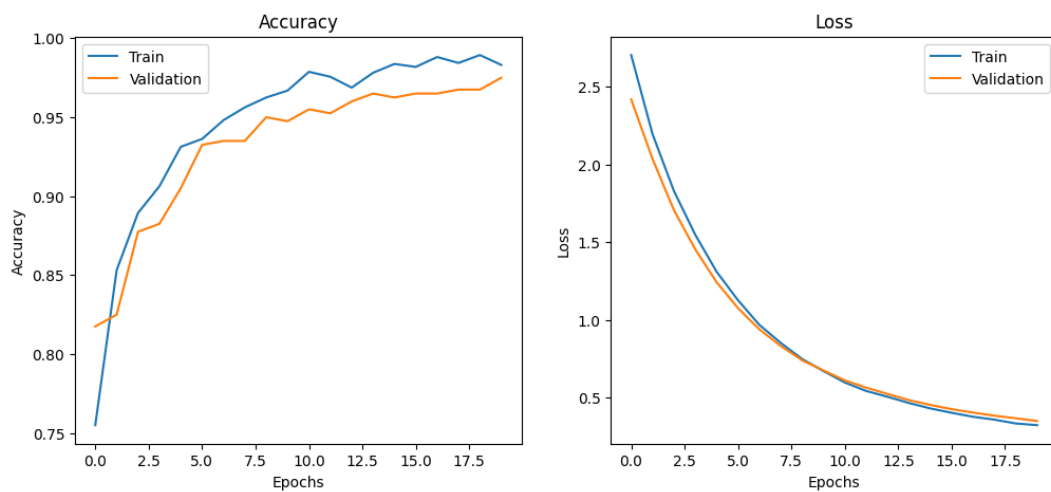


**Fig. 8.2.2** Binary Classification: Cataract vs Normal – Confusion Matrix

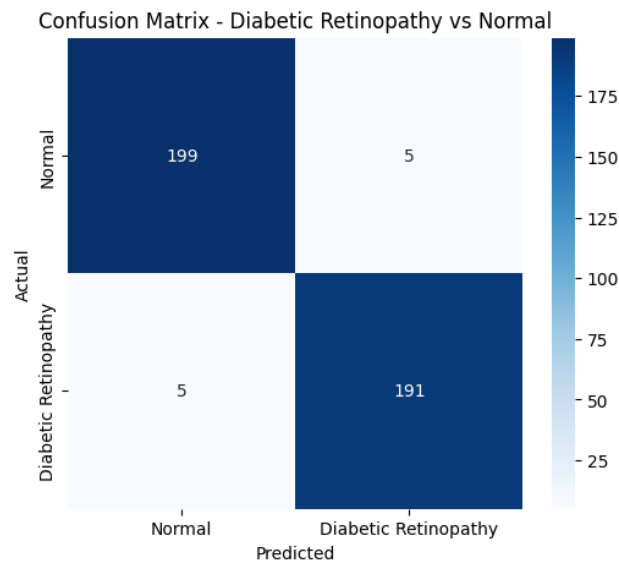
### Diabetic Retinopathy vs Normal

In the diabetic retinopathy classification experiment, the model achieved approximately 99.3% training accuracy and 97.5% validation accuracy. The strong performance demonstrates that the proposed hybrid architecture can successfully identify retinal abnormalities associated with diabetic retinopathy.

The results show that deep feature extraction using ResNet50 combined with residual autoencoder enhancement improves disease detection capability.



**Fig. 8.2.3** Binary Classification: Diabetic Retinopathy vs Normal – Accuracy Plot

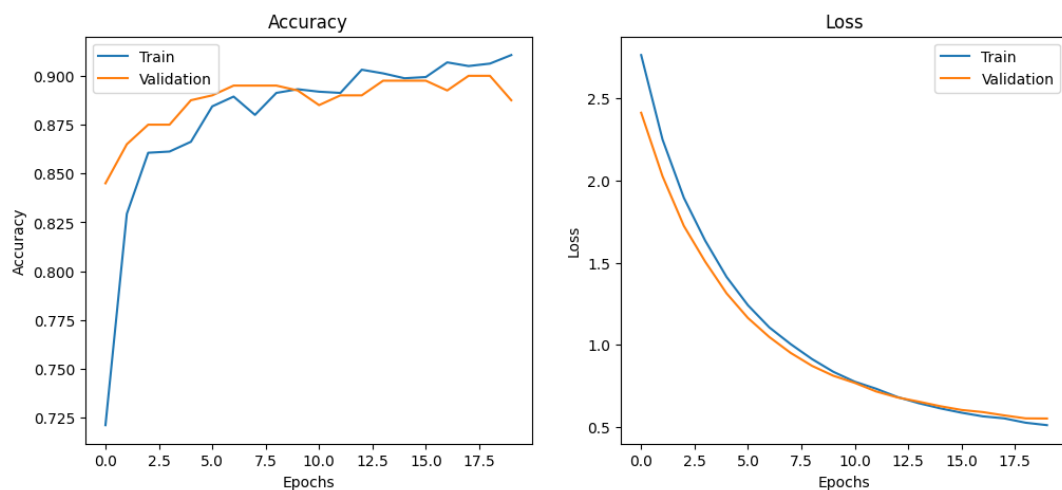


**Fig. 8.2.4** Binary Classification: Diabetic Retinopathy vs Normal – Confusion Matrix

### Glaucoma vs Normal

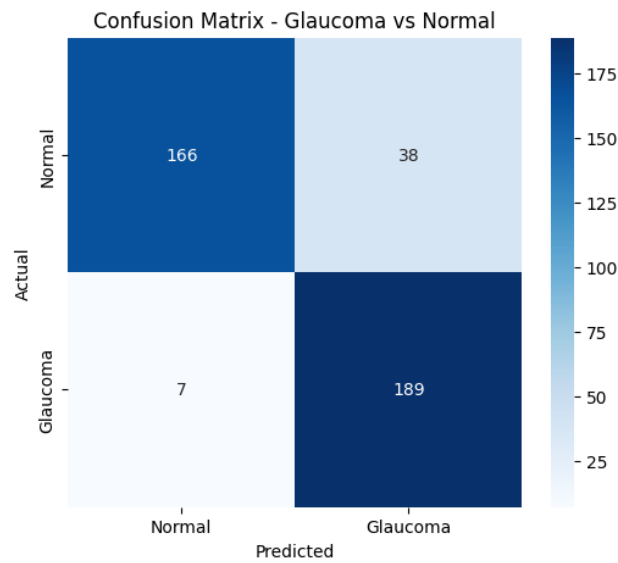
The glaucoma classification experiment achieved approximately 90.0% training accuracy and 88.7% validation accuracy. Compared to the other diseases, glaucoma classification shows slightly lower accuracy due to the subtle visual differences between glaucomatous and normal retinal images.

Despite this challenge, the model still demonstrates strong predictive capability, indicating that the proposed architecture can detect glaucoma-related patterns in retinal images.



**Fig. 8.2.5** Binary Classification: Glaucoma vs Normal – Accuracy Plot





**Fig. 8.2.6** Binary Classification: Glaucoma vs Normal – Confusion Matrix

### Multi-Class Classification Results

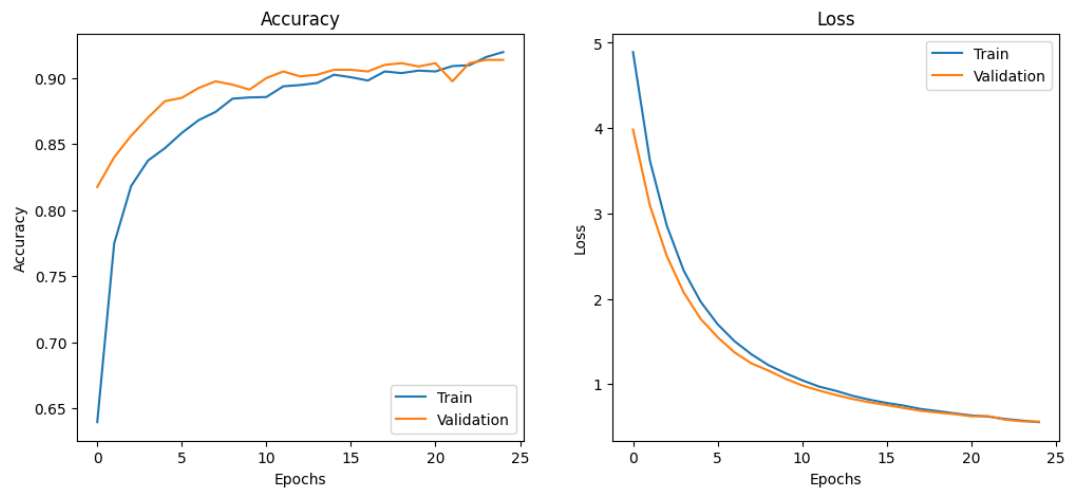
In addition to binary classification experiments, the model was also evaluated using a multi-class classification task involving four retinal image categories: Cataract, Diabetic Retinopathy, Glaucoma, and Normal.

The multi-class model achieved approximately 89–90% validation accuracy, demonstrating the capability of the proposed architecture to classify multiple eye diseases simultaneously.

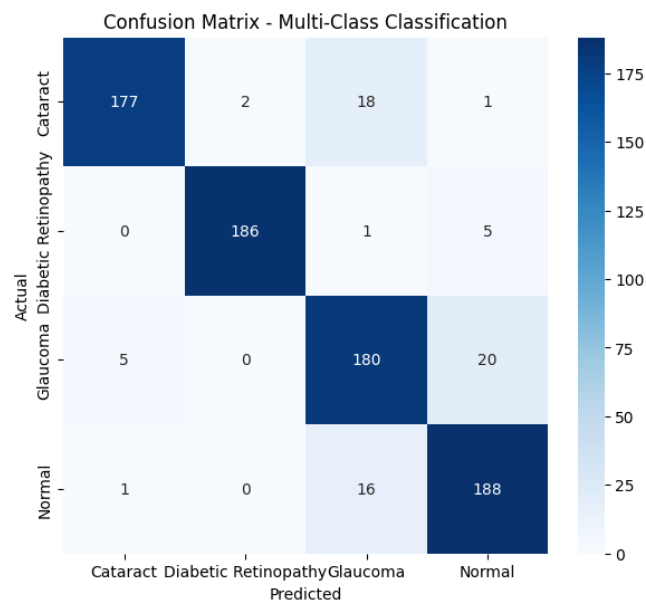
Multi-class classification is inherently more complex because the model must distinguish between multiple disease patterns rather than separating a single disease from normal images. Nevertheless, the obtained accuracy confirms the robustness of the hybrid deep learning architecture.

**Table. 8.2.2** Multi - Class Classification Results

| Class       | Precision | Recall | F1 - Score | Train Acc (%) | Test Acc (%) |
|-------------|-----------|--------|------------|---------------|--------------|
| Cataract    | 0.970     | 0.890  | 0.930      | 92.75         | 91.37        |
| Diabetic    | 0.990     | 0.970  | 0.980      |               |              |
| Retinopathy |           |        |            |               |              |
| Glaucoma    | 0.840     | 0.880  | 0.860      |               |              |
| Normal      | 0.880     | 0.920  | 0.900      |               |              |



**Fig. 8.2.7** Multi - Class Classification – Accuracy Plot



**Fig. 8.2.8** Multi - Class Classification – Confusion Matrix

The experimental results demonstrate that the integration of a residual autoencoder with ResNet50 significantly improves retinal image classification performance. The autoencoder contributes to slight image enhancement by learning residual corrections, while the ResNet50 architecture extracts deep hierarchical features that capture complex disease patterns.

Binary classification experiments achieve very high accuracy, showing that the proposed architecture can reliably distinguish diseased retinal images from normal ones. Multi-class classification performance remains strong despite increased complexity, confirming the robustness of the proposed hybrid model.

Overall, the results indicate that the proposed approach provides an effective solution for automated retinal disease classification and has potential applications in computer-aided medical diagnosis systems.

## **CHAPTER - 9**

### **CONCLUSION AND FUTURE WORK**

#### **9.1 Conclusion**

This project presented a deep learning based approach for automated eye disease classification using a hybrid architecture that integrates a residual autoencoder with the ResNet50 convolutional neural network. The objective of the system was to accurately identify retinal diseases such as Cataract, Diabetic Retinopathy, and Glaucoma from retinal fundus images.

The proposed framework uses a residual autoencoder for slight image enhancement and noise reduction before feature extraction. The refined images are then processed by the ResNet50 deep convolutional network to extract hierarchical features for disease classification. These features are passed through fully connected layers to generate the final prediction.

Experimental results show that the proposed hybrid architecture achieves strong performance in both binary and multi-class classification tasks. High accuracy was observed in distinguishing diseased retinal images from normal ones, particularly for Cataract and Diabetic Retinopathy detection.

The integration of the residual autoencoder with ResNet50 improves feature representation and supports stable model training. Overall, the proposed system demonstrates an effective approach for automated retinal disease detection and highlights the potential of deep learning techniques in computer-aided ophthalmic diagnosis.

## 9.2 Future Work

Although the proposed Autoencoder–ResNet50 based eye disease classification system demonstrates promising performance, there are several directions for further improvement and extension. Future work can focus on enhancing the model architecture and improving dataset diversity to achieve higher classification accuracy and better generalization.

One possible improvement is the use of larger and more diverse retinal image datasets collected from different medical sources. Increasing the dataset size can help the model learn more representative features and improve its ability to generalize to real-world clinical scenarios.

Another potential enhancement is the exploration of advanced deep learning architectures such as EfficientNet, Vision Transformers, or hybrid attention-based networks. These architectures may improve feature extraction and provide better performance for complex medical image classification tasks.

Future research may also include integrating the proposed model into a real-time clinical decision support system. Developing a web or mobile application that allows ophthalmologists to upload retinal images and receive automated diagnostic suggestions could make the system more practical for real-world healthcare applications.

Additionally, further work can focus on improving multi-class classification performance and expanding the system to detect additional eye diseases beyond Cataract, Diabetic Retinopathy, and Glaucoma. Incorporating explainable AI techniques could also help visualize the regions of retinal images that contribute to model predictions, improving transparency and trust in automated diagnostic systems.

Overall, these improvements could make the proposed system more robust, scalable, and suitable for practical deployment in medical imaging applications.

## REFERENCES AND BIBLIOGRAPHY

- [1]. Saqib, Sheikh Muhammad, et al. "Cataract and glaucoma detection based on Transfer Learning using MobileNet." *Heliyon* 10.17 (2024).
- [2]. Mahmood, Salwa Shakir, Sihem Chaabouni, and Ahmed Fakhfakh. "Improving Automated Detection of Cataract Disease through Transfer Learning using ResNet50." *Engineering, Technology & Applied Science Research* 14.5 (2024): 17541-17547.
- [3]. Son, Ki Young, et al. "Deep learning-based cataract detection and grading from slit-lamp and retro-illumination photographs: Model development and validation study." *Ophthalmology Science* 2.2 (2022): 100147.
- [4]. Gomez-Valverde, Juan J., et al. "Automatic glaucoma classification using ´ color fundus images based on convolutional neural networks and transfer learning." *Biomedical optics express* 10.2 (2019): 892-913.
- [5]. Lenka, Satyabrata, Zefree Lazarus Mayaluri, and Ganapati Panda. "Glaucoma detection from retinal fundus images using graph convolution based multi-task model." *e-Prime-Advances in Electrical Engineering, Electronics and Energy* 11 (2025): 100931.
- [6]. Guefrachi, Sarra, Amira Ehtioui, and Habib Hamam. "Automated diabetic retinopathy screening using deep learning." *Multimedia Tools and Applications* 83.24 (2024): 65249-65266.

[7]. Singh, Devendra, and Dinesh C. Dobhal. "A deep learning-based transfer learning approach fine-tuned for detecting diabetic retinopathy." *Procedia Computer Science* 233 (2024): 444-453.

[8]. Hafiyya R.M, Fathima Safna, Fathimath Hanna Mk, Hiba Sherin T, Khadeeja Shehin Ck, 2023, A Deep Learning Approach For Diabetic Retinopathy And Cataract Detection, *INTERNATIONAL JOURNAL OF ENGINEERING RESEARCH & TECHNOLOGY (IJERT)* Volume 11, Issue 01 (June 2023).

[9]. Deepak, G. Divya, and Subraya Krishna Bhat. "Deep learning-based CNN for multiclassification of ocular diseases using transfer learning." *Computer Methods in Biomechanics and Biomedical Engineering: Imaging & Visualization* 12.1 (2024): 2335959.

[10]. Babaqi, Tareq, et al. "Eye disease classification using deep learning techniques." *arXiv preprint arXiv:2307.10501* (2023).

[11]. Abdullah, Ahmed Aizaldeen, Ahmed Aldhahab, and Hanaa Mohsin Al Abboodi. "Deep-Ensemble Learning Models for the Detection and Classification of Eye Diseases Based on Engineering Feature Extraction with Efficientb6 and Densnet169." *International Journal of Intelligent Engineering & Systems* 17.6 (2024).

[12]. Khalaf, Ahmed Tuama, and Salwa Khalid Abdulateef. "Ophthalmic diseases classification based on YOLOv8." *Journal of Robotics and Control (JRC)* 5.2 (2024): 408-415.

[13]. AlMohimeed, Abdulaziz. "Multi-stage framework using transformer models, feature fusion and ensemble learning for enhancing eye disease classification." Scientific Reports 15.1 (2025): 30469.

[14]. Yaurentius, Evelyn Callista, et al. "COMPARATIVE STUDY OF CNNBASED ARCHITECTURES ON EYE DISEASES CLASSIFICATION USING FUNDUS IMAGES TO AID OPHTHALMOLOGIST." (2025).

[15]. Mounika, Satti, and V. RaviSankar. "Multi-Class Classification of Diabetic Retinopathy Diseases From Fundus Images Using Criss-Cross Attention Based Deep Residual Squeeze and Excitation Assisted Vision Transformer Model." International Journal of Imaging Systems and Technology 35.5 (2025): e70199.

[16]. World Health Organization. Strategies for the prevention of blindness in national programmes: a primary health care approach. World health organization, 1997.



## APPENDIX A

### FULL CODE

#### Advanced ResNet50–Autoencoder Framework for Multi-Class Eye Disease Classification

##### Binary Classification: Cataract vs Normal

###### Importing Required Libraries

```
# Import required libraries for deep learning, dataset loading,
ResNet50 transfer learning, and plotting
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing import image_dataset_from_directory
from tensorflow.keras import layers, models, regularizers
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.optimizers import Adam
```

###### Loading Dataset (Only Cataract, Normal)

```
# Load dataset, set training parameters, and create optimized
train/validation data pipelines
DATA_DIR =
"/content/drive/MyDrive/Dataset/Eye_Disease_Classification_Dataset/Cata
ract_Normal"
IMG_SIZE = (224, 224)
BATCH_SIZE = 32
EPOCHS = 20
train_ds = image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="binary"
)
val_ds = image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="binary")
```

```
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.prefetch(AUTOTUNE)
val_ds = val_ds.prefetch(AUTOTUNE)
```

### Residual Autoencoder

# Build a lightweight residual autoencoder to perform slight denoising before feeding images to ResNet

```
def build_autoencoder(img_shape):
    inp = layers.Input(shape=img_shape)
    # small noise correction only
    x = layers.Conv2D(16, 3, activation='relu', padding='same')(inp)
    x = layers.Conv2D(16, 3, activation='relu', padding='same')(x)
    correction = layers.Conv2D(3, 3, padding='same')(x)
    # residual connection
    out = layers.Add()([inp, 0.1 * correction])
    autoencoder = models.Model(inp, out)
    autoencoder.compile(optimizer='adam', loss='mse')
    return autoencoder

autoencoder = build_autoencoder((224,224,3))
```

### Training Autoencoder

# Train autoencoder in an unsupervised manner for denoising, then freeze its weights

```
print("Training Autoencoder...")
autoencoder.fit(
    train_ds.map(lambda x,y: (x,x)),
    epochs=5,
    validation_data=val_ds.map(lambda x,y: (x,x))
)
autoencoder.trainable = False
```

### Final Model (Autoencoder + ResNet50)

# Build final classification model: denoised images from autoencoder → ResNet50 → binary classifier

```
base_model = ResNet50(
    weights="imagenet",
    include_top=False,
    input_shape=(224,224,3)
)
base_model.trainable = False
inputs = layers.Input(shape=(224,224,3))
# denoised image goes to ResNet
x = autoencoder(inputs)
x = base_model(x)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(128, activation="relu",
    kernel_regularizer=regularizers.l2(0.01))(x)
```

```

x = layers.Dropout(0.3)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = models.Model(inputs, outputs)
model.compile(
    optimizer=Adam(1e-4),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

```

### Train Classifier

# Train the final model and store training history for accuracy/loss visualization

```

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS
)

```

### Evaluate Model

# Evaluate model performance on training and validation sets (Cataract vs Normal)

```

train_loss, train_acc = model.evaluate(train_ds)
val_loss, val_acc = model.evaluate(val_ds)
print(f"Training Accuracy (Binary Classification - Cataract vs Normal): {train_acc*100:.2f}%")
print(f"Validation Accuracy (Binary Classification - Cataract vs Normal): {val_acc*100:.2f}%")

```

### Plot Training Curves

# Plot simple training and validation accuracy/loss curves

```

plt.figure(figsize=(12,5))
# Accuracy
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Validation')
plt.title('Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
# Loss
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train')
plt.plot(history.history['val_loss'], label='Validation')
plt.title('Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

```

```
plt.show()
```

### **Additional Metrics**

```
# Compute evaluation metrics (Precision, Recall, F1-score) for Glaucoma  
vs Normal classification
```

```
from sklearn.metrics import classification_report, confusion_matrix
```

```
import pandas as pd
```

```
import seaborn as sns
```

```
# Collect true labels and predictions
```

```
y_true = []
```

```
y_pred = []
```

```
for images, labels in val_ds:
```

```
    preds = model.predict(images)
```

```
    preds = (preds > 0.5).astype("int32").flatten()
```

```
    labels = labels.numpy().astype("int32").flatten()
```

```
    y_true.extend(labels)
```

```
    y_pred.extend(preds)
```

```
# Convert to numpy arrays
```

```
y_true = np.array(y_true)
```

```
y_pred = np.array(y_pred)
```

```
# Classification report (dictionary format)
```

```
report = classification_report(y_true, y_pred, target_names=["Normal",  
"Cataract"], output_dict=True)
```

```
# Convert to DataFrame for table view
```

```
df_report = pd.DataFrame(report).transpose()
```

```
print("\nClassification Report (Table):\n")
```

```
print(df_report)
```

### **Confusion Matrix**

```
# Plot confusion matrix to visualize prediction performance (Cataract  
vs Normal)
```

```
cm = confusion_matrix(y_true, y_pred)
```

```
plt.figure(figsize=(6,5))
```

```
sns.heatmap(
```

```
    cm,
```

```
    annot=True,
```

```
    fmt="d",
```

```
    cmap="Blues",
```

```
    xticklabels=["Normal", "Cataract"],
```

```
    yticklabels=["Normal", "Cataract"]
```

```
)
```

```
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
```

```
plt.title("Confusion Matrix - Cataract vs Normal")
```

```
plt.show()
```

## Binary Classification: Diabetic Retinopathy vs Normal

### Importing Required Libraries

```
# Import required libraries for deep learning, dataset loading,
ResNet50 transfer learning, and plotting
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing import image_dataset_from_directory
from tensorflow.keras import layers, models, regularizers
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.optimizers import Adam
```

### Loading Dataset (Only Diabetic Retinopathy, Normal)

```
# Load dataset, set training parameters, and create optimized
train/validation data pipelines
DATA_DIR =
"/content/drive/MyDrive/4/Dataset/Eye_Disease_Classification_Dataset/Di
abeticRetinopathy_Normal"
IMG_SIZE = (224, 224)
BATCH_SIZE = 32
EPOCHS = 20
train_ds = image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="binary"
)
val_ds = image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="binary"
)
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.prefetch(AUTOTUNE)
val_ds = val_ds.prefetch(AUTOTUNE)
```

### Residual Autoencoder

```
# Build a lightweight residual autoencoder to perform slight denoising
before feeding images to ResNet
```

```

def build_autoencoder(img_shape):
    inp = layers.Input(shape=img_shape)
    # small noise correction only
    x = layers.Conv2D(16, 3, activation='relu', padding='same')(inp)
    x = layers.Conv2D(16, 3, activation='relu', padding='same')(x)
    correction = layers.Conv2D(3, 3, padding='same')(x)
    # residual connection
    out = layers.Add()([inp, 0.1 * correction])
    autoencoder = models.Model(inp, out)
    autoencoder.compile(optimizer='adam', loss='mse')
    return autoencoder
autoencoder = build_autoencoder((224,224,3))

```

### Training Autoencoder

```

# Train autoencoder in an unsupervised manner for denoising, then
freeze its weights
print("Training Autoencoder...")
autoencoder.fit(
    train_ds.map(lambda x,y: (x,x)),
    epochs=5,
    validation_data=val_ds.map(lambda x,y: (x,x))
)
autoencoder.trainable = False

```

### Final Model (Autoencoder + ResNet50)

```

# Build final classification model: denoised images from autoencoder →
ResNet50 → binary classifier
base_model = ResNet50(
    weights="imagenet",
    include_top=False,
    input_shape=(224,224,3)
)
base_model.trainable = False
inputs = layers.Input(shape=(224,224,3))
# denoised image goes to ResNet
x = autoencoder(inputs)
x = base_model(x)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(128, activation="relu",
                  kernel_regularizer=regularizers.l2(0.01))(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = models.Model(inputs, outputs)
model.compile(
    optimizer=Adam(1e-4),
    loss="binary_crossentropy",
    metrics=["accuracy"])

```

### Train Classifier

```
# Train the final model and store training history for accuracy/loss
visualization
```

```
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS
)
```

### Evaluate Model

```
# Evaluate model performance on training and validation sets (Diabetic
Retinopathy vs Normal)
```

```
train_loss, train_acc = model.evaluate(train_ds)
val_loss, val_acc = model.evaluate(val_ds)
print(f"Training Accuracy (Binary Classification - Diabetic Retinopathy
vs Normal): {train_acc*100:.2f}%")
print(f"Validation Accuracy (Binary Classification - Diabetic
Retinopathy vs Normal): {val_acc*100:.2f}%")
```

### Plot Training Curves

```
# Plot simple training and validation accuracy/loss curves
```

```
plt.figure(figsize=(12,5))
# Accuracy
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Validation')
plt.title('Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
# Loss
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train')
plt.plot(history.history['val_loss'], label='Validation')
plt.title('Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

### Additional Metrics

```
# Compute evaluation metrics (Precision, Recall, F1-score) for Glaucoma
vs Normal classification
```

```
from sklearn.metrics import classification_report, confusion_matrix
import pandas as pd
import seaborn as sns
# Collect true labels and predictions
```

```

y_true = []
y_pred = []
for images, labels in val_ds:
    preds = model.predict(images)
    preds = (preds > 0.5).astype("int32").flatten()
    labels = labels.numpy().astype("int32").flatten()
    y_true.extend(labels)
    y_pred.extend(preds)
# Convert to numpy arrays
y_true = np.array(y_true)
y_pred = np.array(y_pred)
# Classification report (dictionary format)
report = classification_report(y_true, y_pred, target_names=["Normal",
"Diabetic Retinopathy"], output_dict=True)
# Convert to DataFrame for table view
df_report = pd.DataFrame(report).transpose()
print("\nClassification Report (Table):\n")
print(df_report)

Confusion Matrix
# Plot confusion matrix to visualize prediction performance (Diabetic
Retinopathy vs Normal)

cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(6,5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Normal", "Diabetic Retinopathy"],
    yticklabels=["Normal", "Diabetic Retinopathy"]
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Diabetic Retinopathy vs Normal")
plt.show()

```



## Binary Classification: Glaucoma vs Normal

### Importing Required Libraries

```
# Import required libraries for deep learning, dataset loading,
ResNet50 transfer learning, and plotting
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing import image_dataset_from_directory
from tensorflow.keras import layers, models, regularizers
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.optimizers import Adam
```

### Loading Dataset (Only Glaucoma, Normal)

```
# Load dataset, set training parameters, and create optimized
train/validation data pipelines
DATA_DIR =
"/content/drive/MyDrive/Dataset/Eye_Disease_Classification_Dataset/Glaucoma_Normal"
IMG_SIZE = (224, 224)
BATCH_SIZE = 32
EPOCHS = 20
train_ds = image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="binary"
)
val_ds = image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="binary"
)
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.prefetch(AUTOTUNE)
val_ds = val_ds.prefetch(AUTOTUNE)
```

### Residual Autoencoder

```
# Build a lightweight residual autoencoder to perform slight denoising
before feeding images to ResNet
```

```

def build_autoencoder(img_shape):
    inp = layers.Input(shape=img_shape)
    # small noise correction only
    x = layers.Conv2D(16, 3, activation='relu', padding='same')(inp)
    x = layers.Conv2D(16, 3, activation='relu', padding='same')(x)
    correction = layers.Conv2D(3, 3, padding='same')(x)
    # residual connection
    out = layers.Add()([inp, 0.1 * correction])
    autoencoder = models.Model(inp, out)
    autoencoder.compile(optimizer='adam', loss='mse')
    return autoencoder
autoencoder = build_autoencoder((224,224,3))

```

### Training Autoencoder

```

# Train autoencoder in an unsupervised manner for denoising, then
freeze its weights
print("Training Autoencoder...")
autoencoder.fit(
    train_ds.map(lambda x,y: (x,x)),
    epochs=5,
    validation_data=val_ds.map(lambda x,y: (x,x))
)
autoencoder.trainable = False

```

### Final Model (Autoencoder + ResNet50)

```

# Build final classification model: denoised images from autoencoder →
ResNet50 → binary classifier
base_model = ResNet50(
    weights="imagenet",
    include_top=False,
    input_shape=(224,224,3)
)
base_model.trainable = False
inputs = layers.Input(shape=(224,224,3))
# denoised image goes to ResNet
x = autoencoder(inputs)
x = base_model(x)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(128, activation="relu",
                  kernel_regularizer=regularizers.l2(0.01))(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = models.Model(inputs, outputs)
model.compile(
    optimizer=Adam(1e-4),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

```

```
)
```

#### **Train Classifier**

```
# Train the final model and store training history for accuracy/loss  
visualization
```

```
history = model.fit(  
    train_ds,  
    validation_data=val_ds,  
    epochs=EPOCHS  
)
```

#### **Evaluate Model**

```
# Evaluate model performance on training and validation sets (Glaucoma  
vs Normal)
```

```
train_loss, train_acc = model.evaluate(train_ds)  
val_loss, val_acc = model.evaluate(val_ds)  
print(f"Training Accuracy (Binary Classification - Glaucoma vs Normal):  
{train_acc*100:.2f}%")  
print(f"Validation Accuracy (Binary Classification - Glaucoma vs  
Normal): {val_acc*100:.2f}%")
```

#### **Plot Training Curves**

```
# Plot simple training and validation accuracy/loss curves  
plt.figure(figsize=(12,5))  
# Accuracy  
plt.subplot(1,2,1)  
plt.plot(history.history['accuracy'], label='Train')  
plt.plot(history.history['val_accuracy'], label='Validation')  
plt.title('Accuracy')  
plt.xlabel('Epochs')  
plt.ylabel('Accuracy')  
plt.legend()  
# Loss  
plt.subplot(1,2,2)  
plt.plot(history.history['loss'], label='Train')  
plt.plot(history.history['val_loss'], label='Validation')  
plt.title('Loss')  
plt.xlabel('Epochs')  
plt.ylabel('Loss')  
plt.legend()  
plt.show()
```

#### **Additional Metrics**

```
# Compute evaluation metrics (Precision, Recall, F1-score) for Glaucoma  
vs Normal classification  
from sklearn.metrics import classification_report, confusion_matrix  
import pandas as pd
```

```

import seaborn as sns
# Collect true labels and predictions
y_true = []
y_pred = []
for images, labels in val_ds:
    preds = model.predict(images)
    preds = (preds > 0.5).astype("int32").flatten()
    labels = labels.numpy().astype("int32").flatten()
    y_true.extend(labels)
    y_pred.extend(preds)
# Convert to numpy arrays
y_true = np.array(y_true)
y_pred = np.array(y_pred)
# Classification report (dictionary format)
report = classification_report(y_true, y_pred, target_names=["Normal",
"Glaucoma"], output_dict=True)
# Convert to DataFrame for table view
df_report = pd.DataFrame(report).transpose()
print("\nClassification Report (Table):\n")
print(df_report)

```

### Confusion Matrix

```

# Plot confusion matrix to visualize prediction performance (Glaucoma
vs Normal)
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(6,5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Normal", "Glaucoma"],
    yticklabels=["Normal", "Glaucoma"]
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Glaucoma vs Normal")
plt.show()

```

## Multi - Class Classification: Cataract vs Diabetic Retinopathy vs Glaucoma vs Normal

### Importing Required Libraries

```
# Import required libraries for deep learning, dataset loading,
ResNet50 transfer learning, and plotting
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing import image_dataset_from_directory
from tensorflow.keras import layers, models, regularizers
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.optimizers import Adam
```

### Loading Dataset (Cataract, Diabetic Retinopathy, Glaucoma, Normal)

```
# Load multi-class dataset (Cataract, Diabetic Retinopathy, Glaucoma,
Normal) and prepare train/validation pipelines
DATA_DIR =
"/content/drive/MyDrive/Dataset/Eye_Disease_Classification_Dataset/C_D_
G_N"
IMG_SIZE = (224, 224)
BATCH_SIZE = 32
EPOCHS = 25
NUM_CLASSES = 4
train_ds = image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="categorical"
)
val_ds = image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="categorical"
)
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.prefetch(AUTOTUNE)
val_ds = val_ds.prefetch(AUTOTUNE)
```

### Residual Autoencoder

# Build a lightweight residual autoencoder to slightly denoise images before passing them to ResNet

```
def build_autoencoder(img_shape):
    inp = layers.Input(shape=img_shape)
    x = layers.Conv2D(16, 3, activation='relu', padding='same')(inp)
    x = layers.Conv2D(16, 3, activation='relu', padding='same')(x)
    correction = layers.Conv2D(3, 3, padding='same')(x)
    # residual connection (very small modification)
    out = layers.Add()([inp, 0.1 * correction])
    autoencoder = models.Model(inp, out)
    autoencoder.compile(optimizer='adam', loss='mse')
    return autoencoder

autoencoder = build_autoencoder((224,224,3))
```

### Training Autoencoder

# Train the autoencoder for image reconstruction (unsupervised) and freeze it before classification

```
print("Training Autoencoder...")
autoencoder.fit(
    train_ds.map(lambda x,y: (x,x)),
    epochs=5,
    validation_data=val_ds.map(lambda x,y: (x,x))
)
autoencoder.trainable = False
```

### Final Model (Autoencoder + ResNet50)

# Build final multi-class model: autoencoder-denoised images → ResNet50 → softmax classifier

```
base_model = ResNet50(
    weights="imagenet",
    include_top=False,
    input_shape=(224,224,3)
)
base_model.trainable = False
inputs = layers.Input(shape=(224,224,3))
# Autoencoder output → ResNet
x = autoencoder(inputs)
x = base_model(x)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(256, activation="relu",
                  kernel_regularizer=regularizers.l2(0.01))(x)
x = layers.Dropout(0.4)(x)
outputs = layers.Dense(NUM_CLASSES, activation="softmax")(x)
model = models.Model(inputs, outputs)
model.compile(
    optimizer=Adam(1e-4),
```

```

        loss="categorical_crossentropy",
        metrics=["accuracy"]
    )

```

### **Train Classifier**

# Train the multi-class model and store history for accuracy and loss visualization

```

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS
)

```

### **Evaluate Model**

# Evaluate model performance for multi-class eye disease classification (Cataract, DR, Glaucoma, Normal)

```

train_loss, train_acc = model.evaluate(train_ds)
val_loss, val_acc = model.evaluate(val_ds)
print(f"Training Accuracy (Multi-Class - Cataract vs Diabetic Retinopathy vs Glaucoma vs Normal): {train_acc*100:.2f}%")
print(f"Validation Accuracy (Multi-Class - Cataract vs Diabetic Retinopathy vs Glaucoma vs Normal): {val_acc*100:.2f}%")

```

### **Plot Training Curves**

# Plot training and validation accuracy/loss curves for multi-class classification

```

plt.figure(figsize=(12,5))
# Accuracy
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Validation')
plt.title('Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
# Loss
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train')
plt.plot(history.history['val_loss'], label='Validation')
plt.title('Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()

```

### **Additional Metrics**

```

# Generate Precision, Recall, and F1-score report for multi-class eye
disease classification

from sklearn.metrics import classification_report, confusion_matrix
import numpy as np
y_true = []
y_pred = []
for images, labels in val_ds:
    preds = model.predict(images)
    preds = np.argmax(preds, axis=1)
    labels = np.argmax(labels.numpy(), axis=1)
    y_true.extend(labels)
    y_pred.extend(preds)
y_true = np.array(y_true)
y_pred = np.array(y_pred)
class_names = ["Cataract", "Diabetic Retinopathy", "Glaucoma",
               "Normal"]
print("\nClassification Report (Multi-Class - C/D/G/N):\n")
print(classification_report(y_true, y_pred, target_names=class_names))

Confusion Matrix
# Plot confusion matrix to visualize prediction performance for multi-
class eye disease classification
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(7,6))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=class_names,
    yticklabels=class_names
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Multi-Class Classification")
plt.show()

model.save("eye_disease_model.h5")
model.save_weights("eye_model.weights.h5")

!pip install ipywidgets

import tensorflow as tf
from tensorflow.keras import layers, models, regularizers
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.optimizers import Adam

```



```

def build_autoencoder(img_shape):
    inp = layers.Input(shape=img_shape)

    x = layers.Conv2D(16, 3, activation='relu', padding='same')(inp)
    x = layers.Conv2D(16, 3, activation='relu', padding='same')(x)

    correction = layers.Conv2D(3, 3, padding='same')(x)
    out = layers.Add()([inp, 0.1 * correction])

    autoencoder = models.Model(inp, out)
    autoencoder.trainable = False

    return autoencoder

autoencoder = build_autoencoder((224,224,3))

# ResNet
base_model = ResNet50(weights="imagenet", include_top=False,
input_shape=(224,224,3))
base_model.trainable = False

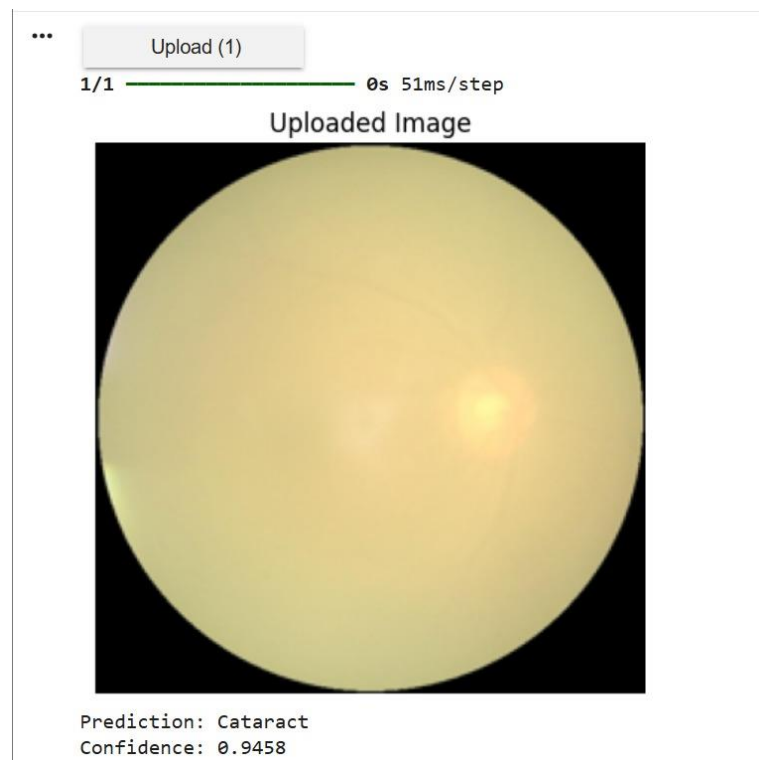
# Final model
inputs = layers.Input(shape=(224,224,3))
x = autoencoder(inputs)
x = base_model(x)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(256, activation="relu",
kernel_regularizer=regularizers.l2(0.01))(x)
x = layers.Dropout(0.4)(x)
outputs = layers.Dense(4, activation="softmax")(x)

model = models.Model(inputs, outputs)

# Compile (needed for loading weights properly)
model.compile(optimizer=Adam(1e-4), loss="categorical_crossentropy",
metrics=["accuracy"])

```

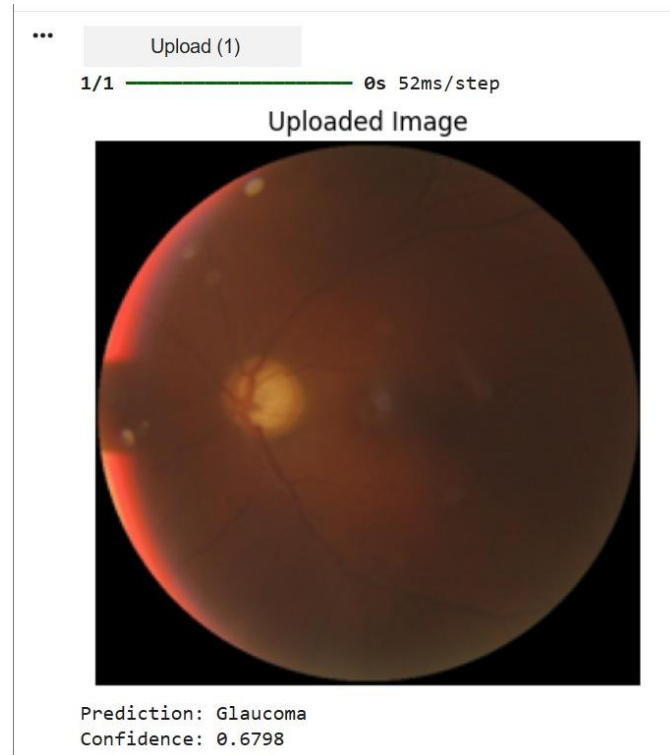
## SCREENSHOTS



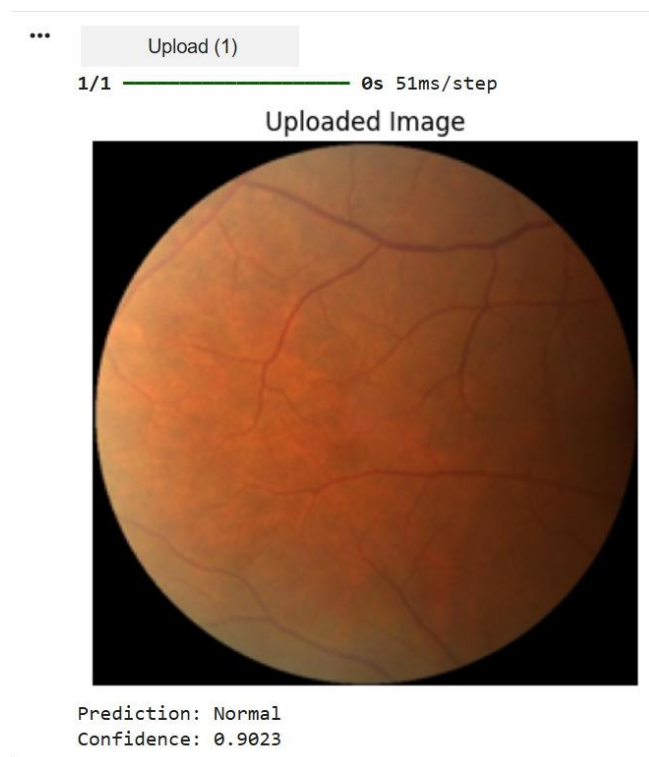
**Figure A.1:** Model Prediction Output for Cataract Retinal Image



**Figure A.2:** Model Prediction Output for Diabetic Retinopathy Retinal Image



**Figure A.3:** Model Prediction Output for Glaucoma Retinal Image



**Figure A.4:** Model Prediction Output for Normal Retinal Image

## APPENDIX B

### Mapping of Sustainable Development Goals (SDGs)

The proposed project on automated eye disease classification using deep learning contributes significantly to global healthcare improvement by enabling early detection of vision threatening diseases such as cataract, diabetic retinopathy, and glaucoma. By leveraging advanced technologies like autoencoders and ResNet50, the system enhances diagnostic accuracy and reduces dependency on manual screening. This aligns with several United Nations Sustainable Development Goals (SDGs), particularly those focused on health, innovation, and accessibility of medical services.

**Table. B.1** Mapping SDGs

| <b>Sustainable Development Goal (SDG)</b>             | <b>Observation</b>                                                                                                                                                            |
|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SDG 3: Good Health and Well-being</b>              | The project supports early detection of eye diseases, helping prevent vision loss and improving overall healthcare outcomes. High accuracy (~98%) ensures reliable diagnosis. |
| <b>SDG 9: Industry, Innovation and Infrastructure</b> | Utilizes advanced deep learning models (Autoencoder + ResNet50) to develop an innovative healthcare solution, promoting technological advancement in medical diagnostics.     |
| <b>SDG 4: Quality Education</b>                       | The project serves as a learning tool for students and researchers in AI, healthcare, and computer vision, enhancing knowledge and skill development.                         |
| <b>SDG 10: Reduced Inequalities</b>                   | Enables affordable and accessible diagnosis, especially in rural or underserved areas where expert ophthalmologists may not be available.                                     |
| <b>SDG 8: Decent Work and Economic Growth</b>         | Reduces workload on medical professionals and improves efficiency in hospitals, contributing to better healthcare service delivery.                                           |

In conclusion, the proposed eye disease classification system demonstrates strong alignment with multiple Sustainable Development Goals by promoting accessible, efficient, and technology-driven healthcare solutions. By improving early diagnosis and reducing healthcare disparities, the project contributes to building a healthier society while fostering innovation and education in the field of artificial intelligence and medical imaging.

Signature of the Guide

## Acceptance Letter



### 3<sup>rd</sup> International Conference on Machine Learning and Autonomous Systems (ICMLAS 2026)

11-13, March 2026 | <https://icmlas.com/2026>

#### Acceptance Letter

| Paper ID   | Paper Title                                                                        | Author's Name                                                                                              |
|------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| ICMLAS-851 | Advanced ResNet50–Autoencoder Framework for Multi-Class Eye Disease Classification | T. Naga Sri Samanvitha, T. Gayathri, V. Gnaneswari, S. Shusruth, Bala Bhaskara Rao Emani, Dr. J. Rama Devi |

Dear Author,

#### Greetings from ICMLAS 2026!!

The Organizing Committee is pleased to inform you that the above peer-reviewed & refereed conference paper has been accepted for presentation at the **3<sup>rd</sup> International Conference on Machine Learning and Autonomous Systems (ICMLAS 2026)** organized by Stamford International University Bangkok, Thailand on 11-13, March 2026.

In this regard, we appreciate if you could send the final paper, IEEE Copyright Form and Payment proof to the conference at the earliest, to ensure a timely publication of your research paper. When submitting your final paper, please highlight the changes made to the research paper according to the specified reviewer comments.

With Thanks,  
Yours' Sincerely

Conference Chair  
Dr. Surekha Lanka  
ICMLAS 2026

# Advanced ResNet50–Autoencoder Framework for Multi-Class Eye Disease Classification

Samanvitha Tadeipalli  
*Department of Computer Science  
and Engineering  
Prasad V. Potluri Siddhartha  
Institute of Technology  
Vijayawada, India  
samtadeipalli@gmail.com*

Shusruth Siddula  
*Department of Computer Science  
and Engineering  
Prasad V. Potluri Siddhartha  
Institute of Technology  
Vijayawada, India  
sushruthsiddula0405@gmail.com*

Gayathri Tati  
*Department of Computer Science  
and Engineering  
Prasad V. Potluri Siddhartha  
Institute of Technology  
Vijayawada, India  
gayathritati367@gmail.com*

Bala Bhaskara Rao Emani  
*Department of Computer Science  
and Engineering  
Prasad V. Potluri Siddhartha  
Institute of Technology  
Vijayawada, India  
balabhaskar605@pvpsiddhartha.ac.in*

Gnaneswari Vemulapalli  
*Department of Computer Science  
and Engineering  
Prasad V. Potluri Siddhartha  
Institute of Technology  
Vijayawada, India  
vemulapallignaneswari5@gmail.com*

Dr. J. Rama Devi  
*Department of Computer Science  
and Engineering  
Prasad V. Potluri Siddhartha  
Institute of Technology  
Vijayawada, India  
k.ramakarthik@gmail.com*

**Abstract**—Glaucoma, diabetic retinopathy, and cataracts are among the most common causes of vision problems causing blindness and loss worldwide. To support early and timely detection, these conditions must be promptly identified and precisely classified. Conventional diagnostic methods are time-consuming, prone to human error, and difficult to apply for large groups because they heavily rely on ophthalmologists. Some well-known deep learning models, such as MobileNet[1] and VGGNet [7], have been used in this field; however, they often encounter difficulties with high computational costs, limited dataset generalization, and reduced accuracy in differentiating between visually similar retinal conditions. In this work, we propose a framework for the classification of eye diseases that combines ResNet50 for the classification process with Autoencoders. The system is meant to be used for both types of classification tasks: binary classification, which checks for conditions like Cataract versus Normal, Diabetic Retinopathy versus Normal, and Glaucoma versus Normal, and multi-class classification, which looks at all these conditions together against Normal. ResNet50 relies on short cut paths, called residual connections, that let the signal flow around some layers. Those paths help stabilize the training and decrease overfitting. A second network, an autoencoder, discards noise and keeps the structures that matter. The cleaned image passes to ResNet50, which picks out useful details to improve feature clarity. Tests on binary but also multi-label sets show higher accuracy, precision and recall than rival models. The joint pipeline shows that an autoencoder paired with ResNet50 gives a promising automated screening framework for early eye disease and can assist ophthalmologists by providing decision support.

**Index Terms**—Eye Disease Classification, Retinal Image Analysis, Cataract, Diabetic Retinopathy, Glaucoma, Convolutional Neural Networks, ResNet50, Autoencoders

## I. INTRODUCTION

Vision is one of the most essential senses for humans, allowing us to see and engage with our surroundings. Sadly,

millions of individuals around the globe experience vision impairment from advancing eye conditions like cataracts, diabetic retinopathy, and glaucoma. The World Health Organization (WHO) states that these conditions represent a considerable proportion of avoidable blindness [16]. Proper and timely identification of these medical conditions together with their correct classification need to happen because they serve as essential requirements for avoiding permanent vision loss while improving patient treatment outcomes. The development of automated systems which can identify and diagnose eye diseases through retinal image analysis has become a leading research area in medical imaging and artificial intelligence technology. Eye disease diagnosis requires ophthalmologists to conduct retinal image examinations through traditional diagnostic methods. The manual assessment process takes a long time because it needs specialized knowledge, while it creates risk from human biases, even though specialists produce accurate assessments. The global population growth together with the decreasing number of available ophthalmologists in multiple regions has made manual screening methods unsuitable because they cannot support large-scale implementation. Researchers have increasing used machine learning together with deep learning methods to create automated systems which can identify medical conditions. Researchers have used multiple existing models for retinal image classification which include MobileNet [1] and VGGNet [7] and InceptionNet [8]. The models display acceptable performance results yet they contain critical weaknesses. Many models require large data sets for good generalization performance, require resources, and often fail to distinguish subtle differences found in conditions such as diabetic retinopathy and Glaucoma. Standard convolutional neural networks (CNNs) face challenges when they process

retinal images with high dimensions because this leads to overfitting and test performance decline on new data sets. This proposed work recommends the combination of the Autoencoder and ResNet50 for eye disease classification in order to overcome the aforementioned shortcomings. Autoencoders have the capability to denoise the images. On the other hand, ResNet50 applies the concept of residual learning for the comprehension of complex patterns in the images and at the same time overcome the difficulty of the vanishing gradient problem. The combination of the above mentioned approaches will make it possible for the system to classify images into both binary and multi-classes. These classifications include the detection of Cataract versus Normal, Diabetic Retinopathy versus Normal, and Glaucoma versus Normal images and Cataract versus Diabetic Retinopathy versus Glaucoma versus Normal images respectively.

The main points and key contributions of the work being presented will be articulated as follows:

- A new classification paradigm can now be developed by integrating ResNet50 and the Autoencoder Fundamentals of ResNet50 and classification.
- The proposed model is used for both Binary and multi-class classification challenges associated with significant eye disorders such as Cataract, Diabetic Retinopathy, Glaucoma.
- Comprehensive comparative study is performed in order to emphasize the improvement of the proposed technique over current deep learning approaches with respect to accuracy, precision, and recall.
- It is described as a scalable and automated tool that can assist ophthalmologists during the screening and decision-making process.

These are the objectives in the light of which this project has been undertaken:

- Design a fully automated deep learning system that is capable of the classifying major eye conditions with retinal images.
- Image denoising with Autoencoders.
- Improvement in the precisions of classification and their reliability with ResNet50.
- To test the model for binary and multi-class classification tasks.
- To support early detection of eye diseases.

In conclusion, this study presents a deep learning-driven eye disease classification method that addresses the shortcomings of current techniques. The combination of Autoencoders and ResNet50 successfully boosts system performance while delivering operational efficiency and scalable capacity, which makes the solution suitable for future medical applications.

## II. LITERATURE REVIEW

Researchers have examined deep learning and transfer learning techniques for eye disease classification. The transfer learning model developed by Sheikh Muhammad Saqib

and his team used MobileNetV1 and MobileNetV2 to detect cataracts and glaucoma. The research showed that MobileNetV1 achieved 90% accuracy for cataracts and 72% accuracy for glaucoma, while MobileNetV2 achieved 87% accuracy for cataracts and 82% accuracy for glaucoma. The system showed diminished performance at around 71% to 72% accuracy when testing the multiclass dataset which included cataract and glaucoma and healthy eye conditions. [1].

Salwa Shakir Mahmood and her team used transfer learning with ResNet50 to improve cataract detection which they compared against CNN and VGG-16. The findings showed that CNN achieved an accuracy of 96.63% while VGG-16 obtained 95.8% and ResNet50 reached 93.5%. The results demonstrated transfer learning capabilities but showed that different architectural designs produced various performance outcomes. [2].

The research team led by Ki Young Son created a deep learning model which detects and grades cataracts using slit-lamp and retro-illumination images. The CNN and region detection networks achieved 98.5% accuracy for slit-lamp images and 92% to 96% accuracy for retro-illumination images which shows that specialized imaging techniques improve detection accuracy. [3]. The study by Juan J. Gomez-Valverde and his team investigated glaucoma classification by using fundus images with CNN and transfer learning. The ensemble CNNs achieved an AUC score of 91.83% while Inception and ResNet reached AUC scores of 98.6% and 96.5% with sensitivities above 95%. The research demonstrated high efficiency but showed that ensemble methods require extensive computational power. [4]. The researchers developed a multi-task glaucoma detection system which combined graph convolutional networks with an improved U-Net method. Their model reached 97.43% accuracy which surpassed the performance of standard CNN models such as AlexNet and GoogleNet. The researchers used SVM and clustering methods which achieved 85 to 93 percent accuracy to demonstrate that their combined approach delivers beneficial outcomes [5]. The researchers employed ResNet152-V2 and InceptionResNet-V2 to screen diabetic retinopathy, achieving screening results of 94.92% and 94.14% [6].

Devendra Singh and Dinesh C. Dobhal used VGG16 and VGG19 to achieve 90.4% accuracy with linear regression and 87.2% with SVM. The findings demonstrate that transfer learning proves effective while showing performance differences which depend on the selected classifier [7].

The research team developed a multi-disease deep learning system which used Google InceptionV3 to identify diabetic retinopathy and cataracts with 99% accuracy. The study presented integrated frameworks as a potential solution while their study results show limitations when applied to multiple datasets [8].

G. Divya Deepak and Subraya Krishna Bhat used lightweight CNN-based transfer learning approaches to improve eye disease classification. Their CNN models achieved 95% and 99.4% and 90% accuracy, which showed that compact models can perform well in resource-constrained situations [9].



The research team led by Tareq Babaqi studied deep learning together with transfer learning methods to classify ocular diseases and achieved 84% accuracy using a CNN baseline, which they increased to 94% through transfer learning. The results demonstrate that pre-trained models provide essential value for ophthalmology [10].

The deep learning models developed by Abdullah and his team utilize EfficientNet-B6 and DenseNet169 as their base models while applying engineered feature extraction to detect eye diseases. The research findings show that using ensemble methods leads to better classification performance yet results in higher demands for processing power [11]. Khalaf and Abdulateef developed a YOLOv8-based framework which enables the classification of ophthalmic diseases and demonstrates that object-detection-based deep learning systems can quickly identify eye diseases [12]. AlMohimeed developed a three-step system which uses transformer models and feature fusion and ensemble learning to improve eye disease diagnosis accuracy which outperforms single model diagnostic techniques [13]. Yaurentius et al. conducted a comparative study of multiple CNN-based architectures for eye disease classification using fundus images which successfully demonstrated the advantages and disadvantages of various deep learning models [14]. Mounika and RaviSankar developed a vision transformer model which uses criss-cross attention and squeeze-and-excitation mechanisms to improve multi-class diabetic retinopathy classification performance through better feature representation [15].

Deep learning and transfer learning methods function as fundamental tools which improve the accuracy of eye disease classification in these studies. Current methods encounter limitations because they produce decreased performance during multiclass processes, and they depend on either specific imaging methods or high computational requirements. The current system needs a solution which can deliver accurate results while maintaining operational efficiency and system adaptability.

### III. METHODOLOGY

#### A. System Overview

The proposed framework for eye disease classification is designed to execute both binary and multi-class classification operations. The model applies Autoencoders to reduce noise and uses the ResNet50 model for classification which establishes a promising and adaptable method to evaluate retinal images.

#### B. Dataset Collection

The images belong to four categories: Cataract, Diabetic Retinopathy, Glaucoma, and Normal (healthy eyes) were sourced from dataset available in Kaggle platform. Care was taken while selecting the images so that all classes were represented in a balanced manner, allowing fair training and evaluation of the proposed model.

Sample images from each class are shown below.

**Cataract:** Cataract refers to the situation where the lens of the eye turns cloudy as shown in Figure 1. This results in the vision being dull and cloudy. This eye complication is dangerous and may cause blindness if left untreated.

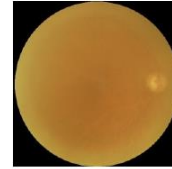


Fig. 1. Sample Retinal Image of Cataract from the Dataset

**Diabetic Retinopathy:** Diabetic Retinopathy (DR) arises as a consequence of diabetes, in which case the high sugar levels in the blood cause damage to the retinal vessels as shown in Figure 2. This leakage in turn, can cause retinal swelling, which may consequently cause vision loss if it is not treated early.



Fig. 2. Sample Diabetic Retinopathy retinal image from the dataset

**Glaucoma:** Glaucoma is a gradual visual disease progressive degeneration of the optic nerve, as shown in Figure 3, which is often caused by intraocular pressure, and can result in irreversible vision loss if undetected.

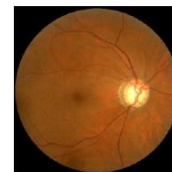


Fig. 3. Sample Glaucoma retinal image from the dataset

**Normal (Healthy Eyes):** Normal retinal images represent healthy eyes without any pathological signs, as shown in Figure 4, serving as the control group for classification.



Fig. 4. Sample Normal retinal image from the dataset

### C. Data Preprocessing

Prior to inputting images into the model, various preprocessing steps are carried out:

- **Resizing:** Every image is adjusted to a consistent size appropriate for ResNet50 input (for instance,  $224 \times 224$  pixels).  
 For each binary classification task in the study, 2,000 retinal images were used, including 400 validation images and 1,600 training images. For three classification tasks: Cataract vs. Normal, Diabetic Retinopathy vs. Normal, and Glaucoma vs. Normal, the split strategy was used by the researchers in the study.  
 For multi-class classification with four classes: Cataract, Diabetic Retinopathy, Glaucoma, and Normal, the study used 4,000 retinal images. An 80:20 split was used in the study, with 3,200 images used for training and 800 images used for validation.

### D. Image Denoising with Autoencoders

The autoencoder functions as a preprocessing module which enhances retinal image quality through noise reduction and preservation of essential structural components. The light residual convolution denoising autoencoder performs a minor image adjustment which maintains the original image resolution. The denoised image is then passed to the ResNet50 model for feature extraction and classification.

- The system uses an lightweight residual convolutional denoising autoencoder to eliminate noise from images while preserving their original resolution.
- The reconstructed outputs conserved significant retinal structures necessary for future classification.
- The denoised images proceed to ResNet50 which performs powerful feature extraction and classification tasks.

### E. Classification using ResNet50

ResNet50 operates as a deep residual architecture that enables learning of complex patterns through its use of residual connections. The structure operates according to the following functions.

- The Autoencoder denoised images serve as input to the system.
- ResNet50 performs feature mapping to create multiple levels of data representation which enables accurate classification.

- The model produces binary labels for each condition versus healthy and multi-class labels for Cataract and Diabetic Retinopathy and Glaucoma and Healthy.

### F. Performance Evaluation

The proposed method evaluation requirement uses standard metrics which measure Accuracy Precision Recall F1-score Confusion Matrix and Accuracy Graphs. The metrics enable complete evaluation of model performance in both binary and multi-class classification systems.

- **Accuracy :** The Accuracy metric measures how many samples were correctly identified from the entire sample set. The assessment shows how accurate the model performs.
- **Precision :** Precision calculates the percentage of correct positive predictions against all samples which the system marked as positive. The system shows how much it can trust its positive predictions.
- **Recall : Recall (or Sensitivity)** assesses how many actual positive cases the system correctly identifies. The system demonstrates its ability to identify actual disease cases.
- **F1-Score :** The F1-score measures Precision and Recall through their harmonic mean to create an equal evaluation of both metrics. The system shows better performance when it works with datasets that have unequal class distribution.
- **Confusion Matrix:** The Confusion Matrix shows all class results by displaying accurate and incorrect class predictions. The system shows its strong points and weak points through this feature.
- **Accuracy Curve:** The Accuracy Curve demonstrates model learning performance by presenting his training results and testing results. The system uses this method to stop overfitting and underfitting problems from developing.

### G. System Architecture

The Eye Disease Classification system design which has been presented uses deep learning technology to divide retinal images into two different groups and multiple different groups. The system will implement Autoencoders together with ResNet50 to identify essential eye diseases.

The proposed architectural design establishes its framework through a base structure that uses a pretrained ResNet50 model together with a compact residual convolutional autoencoder. The autoencoder needs to go through independent training before starting the classification process because it uses shallow convolutional layers for basic image denoising tasks. The system uses the resulting denoised images as input to ResNet50 which reduces overfitting through its frozen initialization method that uses ImageNet weights during training. The global average pooling method combines feature maps from ResNet50 which leads into a fully connected layer that has 128 neurons and dropout regularization. The softmax activation function handles multi-class classification while the sigmoid activation function serves for binary classification in

the final classification layer. The system architecture includes only its top classification layers as trainable components which results in a total of 24 million parameters.

The process involves compiling images obtained from the retina into four categories, namely cataracts, diabetic retinopathy, glaucoma, and normal images. The image undergoes image preprocessing, which involves resizing of images to 224 x 224 pixels, in addition to generating more data from images using rotation, flip, or zoom. To overcome overfitting, image preprocessing helps to improve image clarity.

The Autoencoder system processes the later images. The autoencoder functions as a lightweight residual denoising module which eliminates noise from the image while maintaining essential retinal details and complete image resolution.

The proposed ResNet50-based classification framework first requires retinal images to undergo a denoising process before the actual classification process starts. Three distinct tasks identify Glaucoma and Diabetic Retinopathy and Cataract through binary classification. The multi-class classification system employs one classification branch to identify all four categories which include Cataract and Diabetic Retinopathy and Glaucoma and Normal.

Eventually, the system develops the expected labels for each input image, ensuring the capability of specific disease detection as well as in-depth evaluations. This strategy, in turn, allows ophthalmologists to make fast diagnoses and judgments in their practice due to the flexibility provided.

The overall process will involve preprocessing, denoising, feature extraction and classification as illustrated in Figure 5.

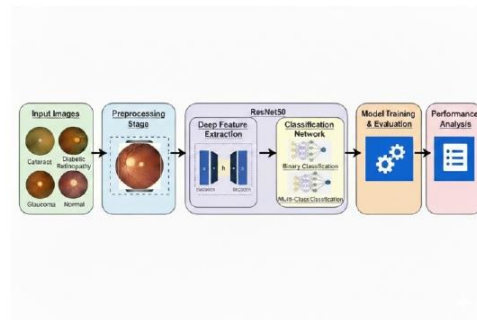


Fig. 5. The suggested system architecture uses ResNet50 and autoencoders to classify eye diseases. The workflow includes preprocessing, feature denoising, representation learning, classification (binary and multi-class), and performance evaluation.

## H. Result Analysis

TABLE I  
BINARY CLASSIFICATION RESULTS FOR EYE DISEASES

| Disease                        | Class                | Precision | Recall | F1-score | Train Acc (%) | Test Acc (%) |
|--------------------------------|----------------------|-----------|--------|----------|---------------|--------------|
| Cataract vs Normal             | Normal               | 0.985     | 0.980  | 0.982    | 97.94         | 98.25        |
|                                | Cataract             | 0.979     | 0.986  | 0.982    |               |              |
| Diabetic Retinopathy vs Normal | Normal               | 0.975     | 0.975  | 0.975    | 99.31         | 97.50        |
|                                | Diabetic Retinopathy | 0.974     | 0.974  | 0.974    |               |              |
| Glaucoma vs Normal             | Normal               | 0.928     | 0.813  | 0.860    | 90.00         | 88.75        |
|                                | Glaucoma             | 0.832     | 0.864  | 0.893    |               |              |

TABLE II  
MULTI-CLASS CLASSIFICATION RESULTS

| Class                | Precision | Recall | F1-score | Train Acc (%) | Test Acc (%) |
|----------------------|-----------|--------|----------|---------------|--------------|
| Cataract             | 0.970     | 0.890  | 0.930    | 92.75         | 91.37        |
| Diabetic Retinopathy | 0.990     | 0.970  | 0.980    |               |              |
| Glaucoma             | 0.840     | 0.880  | 0.860    |               |              |
| Normal               | 0.880     | 0.920  | 0.900    |               |              |

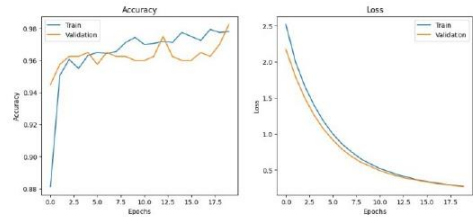


Fig. 6. Binary Classification: Cataract versus Normal – Accuracy Plot

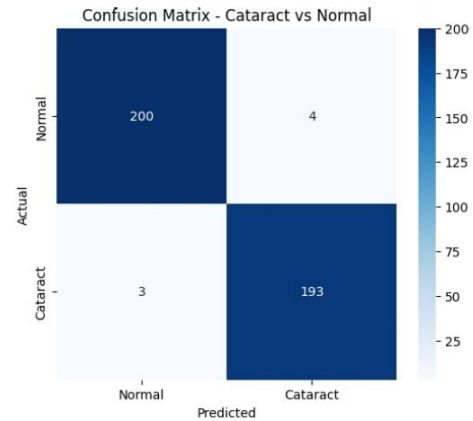


Fig. 7. Binary Classification: Cataract versus Normal – Confusion Matrix

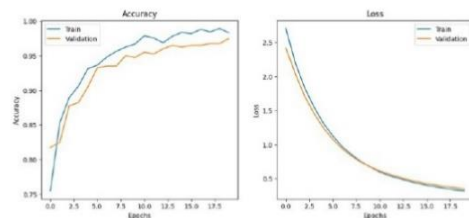


Fig. 8. Binary Classification: Diabetic Retinopathy versus Normal – Accuracy Plot

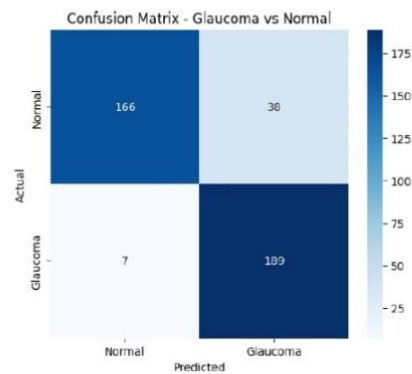


Fig. 11. Binary Classification: Glaucoma v/s Normal – Confusion Matrix

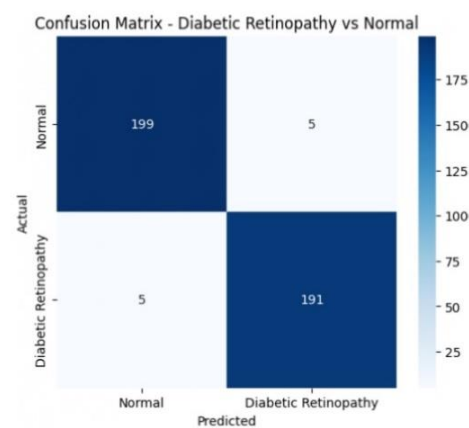


Fig. 9. Binary Classification: Diabetic Retinopathy v/s Normal – Confusion Matrix

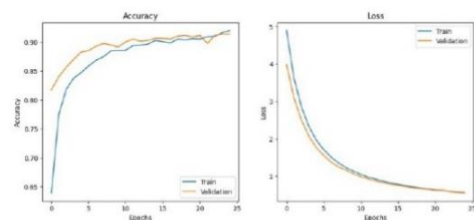


Fig. 12. Multi-Class Classification – Accuracy Plot

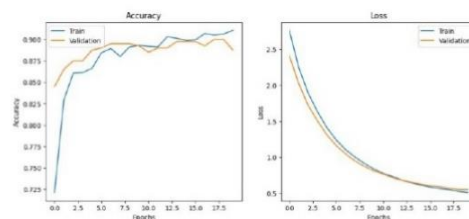


Fig. 10. Binary Classification: Glaucoma v/s Normal – Accuracy Plot

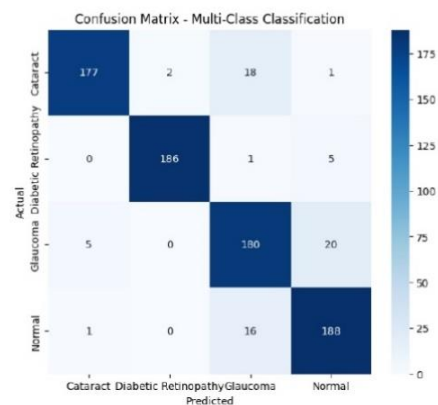


Fig. 13. Multi-Class Classification – Confusion Matrix



**Binary-Class Classification Analysis:** The training and validation accuracy trends for binary classification tasks are shown in Figures 6, 8, and 10, while the corresponding confusion matrices are presented in Figures 7, 9, and 11.

The classification tasks for Cataract and Diabetic Retinopathy show high precision and recall results which demonstrate that researchers can clearly distinguish between diseased and healthy retinal images. The Glaucoma versus Normal classification demonstrates lower recall performance according to Table I and the confusion matrix shown in Figure 11 compared to the other binary classification tasks. The feature overlap between these classes creates a higher probability of making incorrect predictions which demonstrates the fundamental difficulty of detecting glaucoma through fundus images.

**Multi-Class Classification Analysis:** The performance of the multi-class classification model is visualized using the accuracy plot in Figure 12 and the confusion matrix in Figure 13.

The model performance evaluation used validation predictions from a fixed train-validation split to assess the reliability of reported results. The classification performance showed consistent results because the accuracy and precision and recall measurements maintained stability throughout the experiments.

#### LIMITATIONS AND CHALLENGES

The experimental evaluation in this study is based on a fixed train-validation split, and the reported performance metrics are obtained from this setup. The results show consistent trends across the different classification tasks; however, formal statistical significance testing and confidence interval estimation were not performed. Future research will utilize cross-validation and statistical testing methods. The model's accuracy depends on dataset quality and dataset diversity because different image capture conditions and lighting conditions and resolution changes affect its prediction performance. The dataset achieves balance yet it fails to show all clinical variations that exist among different population groups. The first challenge emerges when visual characteristics of different conditions show similarities, which results in incorrect identification of early-stage glaucoma and normal eye conditions, as reflected in the comparatively lower glaucoma classification performance observed in the experimental results. The system needs enhanced performance because its existing offline capabilities require faster operation and better integration with hospital systems and more research validation using larger multi-institutional datasets. The system depends on GPU resources for its training process, which creates scalability problems that will impact healthcare settings which have restricted access to resources.

#### CONCLUSION AND FUTURE SCOPE

The research established an automatic deep learning system which utilizes Autoencoders in combination with ResNet50 to achieve precise eye disease classification through its binary and multiple class identification capabilities. The research

shows that the proposed system enables ophthalmologists to identify eye conditions and make treatment decisions which may help reduce the risk of vision loss. Upcoming research needs to advance clinical relevance through two main objectives which include developing a dataset with diverse sample types and testing the model across different imaging systems and developing a system that medical staff can use in remote areas. The proposed approach shows potential as a user-friendly solution for widespread eye disease screening and remote ophthalmology services through the integration of explainable artificial intelligence with portable diagnostic equipment.

#### REFERENCES

- [1] Saqib, Sheikh Muhammad, et al. "Cataract and glaucoma detection based on Transfer Learning using MobileNet." *Heliyon* 10.17 (2024).
- [2] Mahmood, Salwa Shakir, Sihem Chaabouni, and Ahmed Fakhfakh. "Improving Automated Detection of Cataract Disease through Transfer Learning using ResNet50." *Engineering, Technology & Applied Science Research* 14.5 (2024): 17541-17547.
- [3] Son, Ki Young, et al. "Deep learning-based cataract detection and grading from slit-lamp and retro-illumination photographs: Model development and validation study." *Ophthalmology Science* 2.2 (2022): 100147.
- [4] Gomez-Valverde, Juan J., et al. "Automatic glaucoma classification using color fundus images based on convolutional neural networks and transfer learning." *Biomedical optics express* 10.2 (2019): 892-913.
- [5] Lenka, Satyabrata, Zefrey Lazarus Mayaluri, and Ganapati Panda. "Glaucoma detection from retinal fundus images using graph convolution based multi-task model." *e-Prime-Advances in Electrical Engineering, Electronics and Energy* 11 (2025): 100931.
- [6] Guefrachi, Sarra, Amira Echhoui, and Habib Hamam. "Automated diabetic retinopathy screening using deep learning." *Multimedia Tools and Applications* 83.24 (2024): 65249-65266.
- [7] Singh, Devendra, and Dinesh C. Dobhal. "A deep learning-based transfer learning approach fine-tuned for detecting diabetic retinopathy." *Procedia Computer Science* 233 (2024): 444-453.
- [8] Hafiyya R.M, Fathima Safna, Fathimath Hanna Mk, Hiba Sherin T, Khadeja Shehin Ck, 2023, A Deep Learning Approach For Diabetic Retinopathy And Cataract Detection, *INTERNATIONAL JOURNAL OF ENGINEERING RESEARCH & TECHNOLOGY (IJERT)* Volume 11, Issue 01 (June 2023).
- [9] Deepak, G. Divya, and Subraya Krishna Bhat. "Deep learning-based CNN for multiclassification of ocular diseases using transfer learning." *Computer Methods in Biomechanics and Biomedical Engineering: Imaging & Visualization* 12.1 (2024): 2335959.
- [10] Babaei, Tareq, et al. "Eye disease classification using deep learning techniques." *arXiv preprint arXiv:2307.10501* (2023).
- [11] Abdullah, Ahmed Aizaldeen, Ahmed Aldhabab, and Hanaa Mohsin Al Abboodi. "Deep-Ensemble Learning Models for the Detection and Classification of Eye Diseases Based on Engineering Feature Extraction with Efficientb6 and Densnet169." *International Journal of Intelligent Engineering & Systems* 17.6 (2024).
- [12] Khalaf, Ahmed Tuama, and Salwa Khalid Abdulateef. "Ophthalmic diseases classification based on YOLOv8." *Journal of Robotics and Control (JRC)* 5.2 (2024): 408-415.
- [13] AlMohimeed, Abdulaziz. "Multi-stage framework using transformer models, feature fusion and ensemble learning for enhancing eye disease classification." *Scientific Reports* 15.1 (2025): 30469.
- [14] Yauentius, Evelyn Callista, et al. "COMPARATIVE STUDY OF CNN-BASED ARCHITECTURES ON EYE DISEASES CLASSIFICATION USING FUNDUS IMAGES TO AID OPHTHALMOLOGIST." (2025).
- [15] Mounika, Satti, and V. RaviSankar. "Multi-Class Classification of Diabetic Retinopathy Diseases From Fundus Images Using Criss-Cross Attention Based Deep Residual Squeeze and Excitation Assisted Vision Transformer Model." *International Journal of Imaging Systems and Technology* 35.5 (2025): e70199.
- [16] World Health Organization. *Strategies for the prevention of blindness in national programmes: a primary health care approach*. World health organization, 1997.